

Lecture 7

Linear Regression

Assigned reading (this class and next class):

1.25, 1.26 (regularization, model selection)—assigned in Lecture #3

4.1.6 (Regularization for linear regression)

2.1.3 (Bayes Theorem)

2.6 (Bayesian modeling and regularization)

9.2.2 (Lasso regularization)—spillover into next class

Roadmap

- Linear Regression
 - Finish Probabilistic/MLE view
 - Geometric view
 - Regularization

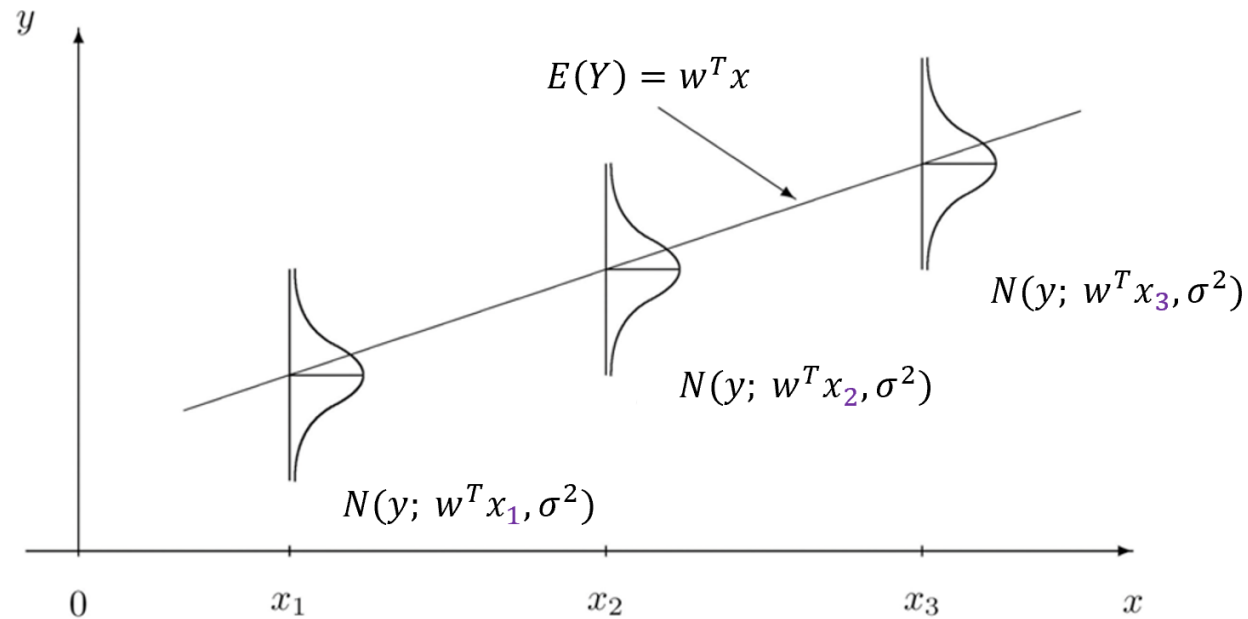
Roadmap

- Linear Regression
 - Finish Probabilistic/MLE view
 - Geometric view
 - Regularization

Recap from last class:

Gaussian linear regression, $p(y|x) = N(y|w^T x, \sigma^2)$

For every value $X = x$, the target variable, Y , takes on a Gaussian distribution with the same variance, σ^2 :



Recap from last class:

“Training” a Gaussian linear regression model

How will we fit the regression model, $p(y|x) = N(y|w^T x, \sigma^2)$?

$$\begin{aligned}\text{MLE: } \theta_{MLE} &= (w_{MLE}, \sigma_{MLE}^2) = \arg \max_{(w, \sigma^2)} \log p(D = \{(x_i, y_i)_{i=1}^n\} | \theta) \\ &= \arg \max_{(w, \sigma^2)} \sum_{i=1}^n \log p(y_i | x_i, \theta) \\ &= \arg \max_{(w, \sigma^2)} \sum_{i=1}^n \log N(y_i | w^T x_i, \sigma^2) \\ &= \arg \max_{(w, \sigma^2)} \sum_{i=1}^n \log \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - w^T x_i)^2 \\ &= \arg \max_{(w, \sigma^2)} n \log \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - w^T x_i)^2\end{aligned}$$

Recap from last class:

“Training” a Gaussian linear regression model

$$p(y|x) = N(y|w^T x, \sigma^2)$$

Note, if we ignore σ_{MLE}^2

$$(w_{MLE}, \sigma_{MLE}^2) = \arg \max_{(w, \sigma^2)} n \log \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - w^T x_i)^2$$

... then estimating w above is the same as

$$\begin{aligned} &= \arg \min_w \sum_{i=1}^n (y_i - w^T x_i)^2 \\ &= \arg \min_w \sum_{i=1}^n (y_i - \hat{y}_i)^2 \end{aligned}$$

“least squares” loss function!

i.e., least squares arises naturally from conditional Gaussian MLE

Recap from last class:

"Training" a Gaussian linear regression model

$$p(y|x) = N(y|w^T x, \sigma^2)$$

Note, if we ignore σ_{MLE}^2

$$(w_{MLE}, \sigma_{MLE}^2) = \arg \max_{(w, \sigma^2)} n \log \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - w^T x_i)^2$$

... then estimating w above is the same as

$$= \arg \min_w \sum_{i=1}^n (y_i - w^T x_i)^2$$

$$= \arg \min_w \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

"vectorize" our objective

$$\begin{pmatrix} y_1 - w^T x_1 \\ \vdots \\ y_n - w^T x_n \end{pmatrix} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} - \begin{pmatrix} x_1^T w \\ \vdots \\ x_n^T w \end{pmatrix} = y - \begin{pmatrix} x_1^T \\ \vdots \\ x_n^T \end{pmatrix} w = y - Aw$$

"least squares" loss function!

i.e., least squares arises naturally from conditional Gaussian MLE

$$= \arg \min_w (y - Aw)^T (y - Aw)$$

Recap from last class:

"Training" a Gaussian linear regression model

So want to minimize this wrt w :

$$\begin{aligned}\mathcal{L} &= \|y - Aw\|_2^2 && (y \in \mathbb{R}^n, A \in \mathbb{R}^{n \times d}, w \in \mathbb{R}^{d \times 1}) \\ &= (y - Aw)^T (y - Aw) \\ &= (y^T - (Aw)^T)(y - Aw) \\ &= y^T y - w^T A^T y - y^T A w + w^T A^T A w \\ &= y^T y - 2w^T A^T y + w^T A^T A w\end{aligned}$$

$$\begin{pmatrix} x_1^T \\ \vdots \\ x_n^T \end{pmatrix}$$

A

To minimize, we want to set $\frac{\partial \mathcal{L}}{\partial w} = 0$.

Most easily achieved by using the rules of vector calculus, so lets do a quick refresher.

$$p(y|x) = N(y|w^T x, \sigma^2)$$

Refresher on vector calculus

Some “rules” for taking gradients with respect to vectors.

- e.g., for vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^{d \times 1}$, such that $\mathbf{a}^T \mathbf{b} \in \mathbb{R}$,

$$\frac{\partial(\mathbf{a}^T \mathbf{b})}{\partial a_j} = \frac{\partial(a_1 b_1 + a_2 b_2 + \cdots a_d b_d)}{\partial a_j} = b_j \in \mathbb{R}$$

Thus,

$$\frac{\partial(\mathbf{a}^T \mathbf{b})}{\partial \mathbf{a}} = \frac{\partial(a_1 b_1 + a_2 b_2 + \cdots a_d b_d)}{\partial \mathbf{a}} = \mathbf{b} \in \mathbb{R}^{d \times 1}$$

(not true for $\mathbf{a} \mathbf{b}^T$ which is a matrix, be careful!)

$$= \frac{\partial(\mathbf{b}^T \mathbf{a})}{\partial \mathbf{a}}$$

Refresher on vector calculus

For vector $x \in \mathbb{R}^{d \times 1}$, and matrix $\Sigma \in \mathbb{R}^{d \times d}$

$$\frac{\partial x^T \Sigma x}{\partial x} = (\Sigma + \Sigma^T)x$$

Thus, if Σ is symmetric such that $\Sigma = \Sigma^T$ then

$$\frac{\partial x^T \Sigma x}{\partial x} = 2\Sigma x$$

(similar to the scalar version: $\frac{\partial(ax^2)}{\partial x} = 2ax$)

"Training" a Gaussian linear regression model

So want to minimize

$$\mathcal{L} = (y - Aw)^T (y - Aw)$$

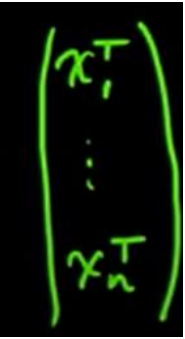
$$(y \in \mathbb{R}^n, A \in \mathbb{R}^{n \times d}, w \in \mathbb{R}^{d \times 1})$$

$$= (y^T - (Aw)^T)(y - Aw)$$

$$= y^T y - w^T A^T y - y^T A w + w^T A^T A w$$

$$= y^T y - 2w^T A^T y + w^T A^T A w$$

To minimize, we want to set $\frac{\partial \mathcal{L}}{\partial w^T} = 0$.



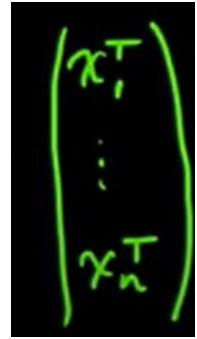
A handwritten green matrix notation for A , showing a column vector of x_1^T , a vertical ellipsis, and x_n^T . To the right, in purple, is the text $= A \in \mathbb{R}^{n \times d}$.

"Training" a Gaussian linear regression model

So want to minimize

$$\mathcal{L} = (y - Aw)^T (y - Aw)$$

$$(y \in \mathbb{R}^n, A \in \mathbb{R}^{n \times d}, w \in \mathbb{R}^{d \times 1})$$



Handwritten green notation: $\begin{pmatrix} x_1^T \\ \vdots \\ x_n^T \end{pmatrix}$. To its right, handwritten purple text: $= A \in \mathbb{R}^{n \times d}$.

$$= (y^T - (Aw)^T)(y - Aw)$$

$$= y^T y - w^T A^T y - y^T A w + w^T A^T A w$$

$$= y^T y - 2w^T A^T y + w^T A^T A w$$

To minimize, we want to set $\frac{\partial \mathcal{L}}{\partial w^T} = 0$.

$$\nabla_w \mathcal{L} = \begin{bmatrix} \partial \mathcal{L} / \partial w_1 \\ \partial \mathcal{L} / \partial w_2 \\ \vdots \\ \partial \mathcal{L} / \partial w_d \end{bmatrix} = \begin{bmatrix} -2A^T y + 2A^T A w \end{bmatrix}$$
$$= \underbrace{(-A^T y + A^T A w)}_{d \times 1} \cdot 2$$

"Training" a Gaussian linear regression model

So want to minimize

$$\begin{aligned}\mathcal{L} &= (y - Aw)^T (y - Aw) \\ &= (y^T - (Aw)^T)(y - Aw) \\ &= y^T y - w^T A^T y - y^T A w + w^T A^T A w \\ &= y^T y - 2w^T A^T y + w^T A^T A w\end{aligned}$$

To minimize, we want to set $\frac{\partial \mathcal{L}}{\partial w} = 0$.

$$\nabla_w \mathcal{L} = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial w_1} \\ \frac{\partial \mathcal{L}}{\partial w_2} \\ \vdots \\ \frac{\partial \mathcal{L}}{\partial w_n} \end{bmatrix} = \begin{bmatrix} -2A^T y + 2A^T A w \end{bmatrix}$$
$$= \underbrace{(-A^T y + A^T A w)}_{dx1} \cdot 2$$

"left pseudo-inverse" of A
 $(A^T A)^{-1} A^T * A = I$

$$A^+ A = I$$

$$\begin{pmatrix} x_1^T \\ \vdots \\ x_n^T \end{pmatrix} = A \in \mathbb{R}^{n \times d}$$

set to 0

rectangular
so no A^{-1} !

$$-A^T y + A^T A w = 0$$

$$A^T A w = A^T y$$

$$w = (A^T A)^{-1} A^T y$$

"Training" a Gaussian linear regression model

- A^+ is the Moore-Penrose inverse.
- Can be used even when A is not full rank.
- i.e., when A has dependent feature vectors.
- Will yield w_{MLE} with min. 2-norm, $\|w_{MLE}\|_2$

"left pseudo-inverse" of A
 $(A^T A)^{-1} A^T A = I$

$$A^+ A = I$$

$$\begin{pmatrix} x_1^T \\ \vdots \\ x_n^T \end{pmatrix} = A \in \mathbb{R}^{n \times d}$$

rectangular
so no A^{-1} !

$$-A^T y + A^T A w = 0$$

$$A^T A w = A^T y$$

$$w = (A^T A)^{-1} A^T y$$

"Training" a Gaussian linear regression model

- We still need to check if the critical point, $w = (A^T A)^{-1} A^T y$ is minimum of the squared error loss.
 - Recall $\nabla_w \mathcal{L} = -2A^T y + 2A^T A w$
 - So Hessian matrix ($\nabla_w^2 \mathcal{L}$) is $2A^T A$. When is $A^T A$ PD?
 - When the features in data set are independent (when it has full rank).
- ❖ N.B. σ^2 from MLE is just the mean squared residual, $\sigma^2 = \frac{1}{N} \sum_i (y_i - w^T x)^2$.

$$\begin{pmatrix} x_1^T \\ \vdots \\ x_n^T \end{pmatrix} = A \in \mathbb{R}^{n \times d}$$

$$\nabla_w \mathcal{L} = \begin{bmatrix} \partial \mathcal{L} / \partial w_1 \\ \partial \mathcal{L} / \partial w_2 \\ \vdots \\ \partial \mathcal{L} / \partial w_d \end{bmatrix} = \begin{bmatrix} -2A^T y + 2A^T A w \end{bmatrix}$$
$$= \underbrace{-A^T y + A^T A w}_{dx1}$$

$$\begin{aligned} -A^T y + A^T A w &= 0 \\ A^T A w &= A^T y \\ \boxed{w} &= (A^T A)^{-1} A^T y \end{aligned}$$

"Training" a Gaussian linear regression model

- We still need to check if the critical point, $w = (A^T A)^{-1} A^T y$ is minimum of the squared error loss.

$$\begin{pmatrix} x_1^T \\ \vdots \\ x_n^T \end{pmatrix} = A \in \mathbb{R}^{n \times d}$$

- Recall $\nabla_w \mathcal{L} = -2A^T y + 2A^T A w$

- So Hessian matrix is $2A^T A$ (assuming A is full rank).
- When the feature matrix A is full rank).

❖ N.B. σ^2 from MLE is just the mean squared residual, $\sigma^2 = \frac{1}{N} \sum_i (y_i - w^T x)^2$.

$$\nabla_w \mathcal{L} = \begin{bmatrix} \partial \mathcal{L} / \partial w_1 \\ \partial \mathcal{L} / \partial w_2 \\ \vdots \\ \partial \mathcal{L} / \partial w_d \end{bmatrix} = \begin{bmatrix} -2A^T y + 2A^T A w \end{bmatrix}$$
$$= \underbrace{-A^T y + A^T A w}_{dx1}$$

$$\begin{aligned} -A^T y + A^T A w &= 0 \\ A^T A w &= A^T y \\ \boxed{w} &= \boxed{(A^T A)^{-1} A^T y} \end{aligned}$$

Using basis expansions instead of x_i

$$\begin{pmatrix} x_1^T \\ \vdots \\ x_n^T \end{pmatrix} = A$$

$$w = (A^T A)^{-1} A^T y$$

- $\{x_j\} \rightarrow \{\Phi(x_j)\}$?
- Just define A with $\Phi^T(x_j)$ instead of x_j because $\Phi(x)$ is fixed ahead of time, so its like someone just gave us different raw inputs x .

$$x' = \Phi(x) = \Phi([x_1, x_2]^T) = [1, x_1, x_2, x_1 x_2, x_1^2, x_2^2]^T \in \mathbb{R}^{6 \times 1}$$

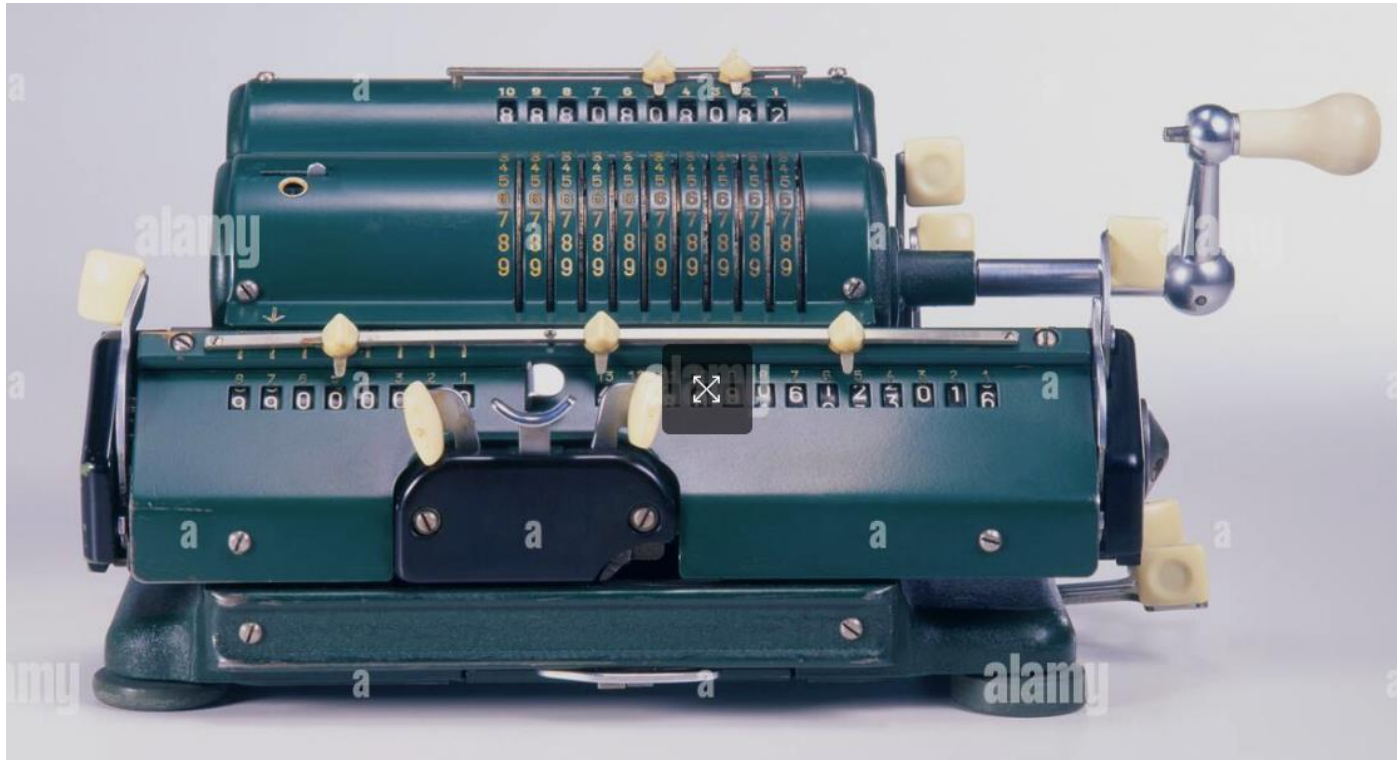
Polynomial expansion of order 2 (i.e. quadratic)

$$\begin{aligned} w &= (\Phi(X)^T \phi(X))^{-1} \phi(X)^T y \\ &= X'^T X')^{-1} X'^T y \end{aligned}$$

$$Y = \phi(X)w = x'w$$

Regression in 1950s

- Electromechanical desk "calculators" were used.
- Could take up to 24 hours to receive the result from one regression on a small data set.

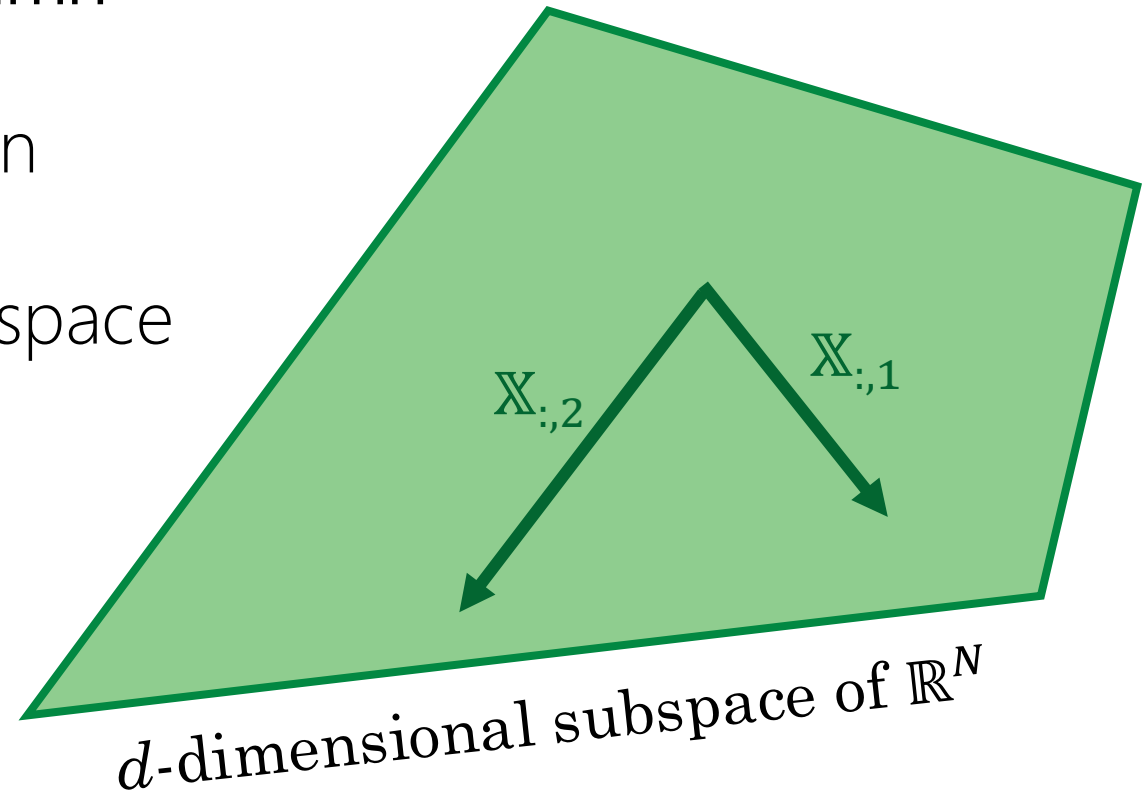


Roadmap

- Linear Regression
 - Finish Probabilistic/MLE view
 - Geometric view
 - Regularization

[Linear Algebra] Span $\mathbf{X} \in \mathbb{R}^{N \times d}$

- The set of all possible linear combinations of the columns of $\mathbf{X} \in \mathbb{R}^{N \times d}$ is called the **span** of $\mathbf{X}$ (denoted $\text{span}(\mathbf{X})$), also called the **column space**.
- Intuitively, $\text{span}(\mathbf{X})$ is all n -vectors you can "reach" using the columns of $\mathbf{X}$.
- $\text{span}(\mathbf{X})$ is a (at most) d -dimensional subspace of $\mathbb{R}^N$.
- dimension of $\text{span}(\mathbf{X})$ is $\text{rank}(\mathbf{X}) \leq d$



[Linear Algebra] Matrix-Vector Multiplication

- **Approach 1:** So far, we've thought of our model as horizontally stacked predictions per datapoint:

$$\begin{bmatrix} | \\ \mathbb{Y} \\ | \end{bmatrix} = \begin{bmatrix} \text{---} & \mathbf{x}_1^T & \text{---} \\ \text{---} & \mathbf{x}_2^T & \text{---} \\ & \vdots & \\ \text{---} & \mathbf{x}_N^T & \text{---} \end{bmatrix} \begin{bmatrix} w \end{bmatrix}$$

Matrix sizes:

$$N \times 1 = (N \times d) (d \times 1)$$

$$Y = Aw = Xw$$

- **Approach 2:** Helpful to think of matrix-vector multiplication as a **linear combination of feature vectors**, scaled by **parameters**.

[Linear Algebra] Matrix-Vector Multiplication

- **Approach 1:** So far, we've thought of our model as horizontally stacked predictions per datapoint:

$$\begin{bmatrix} | \\ \mathbb{Y} \\ | \end{bmatrix} = \begin{bmatrix} \text{---} & \mathbf{x}_1^T & \text{---} \\ \text{---} & \mathbf{x}_2^T & \text{---} \\ & \vdots & \\ \text{---} & \mathbf{x}_N^T & \text{---} \end{bmatrix} \begin{bmatrix} w \end{bmatrix}$$

Matrix sizes:

$$N \times 1 = (N \times d) (d \times 1)$$

$$Y = Aw = \mathbb{X}w$$

- **Approach 2:** Helpful to think of matrix-vector multiplication as a **linear combination of feature vectors**, scaled by **parameters**.

$$\begin{bmatrix} | \\ \mathbb{Y} \\ | \end{bmatrix} = \begin{bmatrix} | & & | \\ \mathbb{X}_{:,1} & \dots & \mathbb{X}_{:,d} \\ | & & | \end{bmatrix} \begin{bmatrix} w \end{bmatrix} = w_1 \mathbb{X}_{:,1} + w_2 \mathbb{X}_{:,2} + \dots + w_d \mathbb{X}_{:,d} = \sum_j w_j \mathbb{X}_{:,j}$$

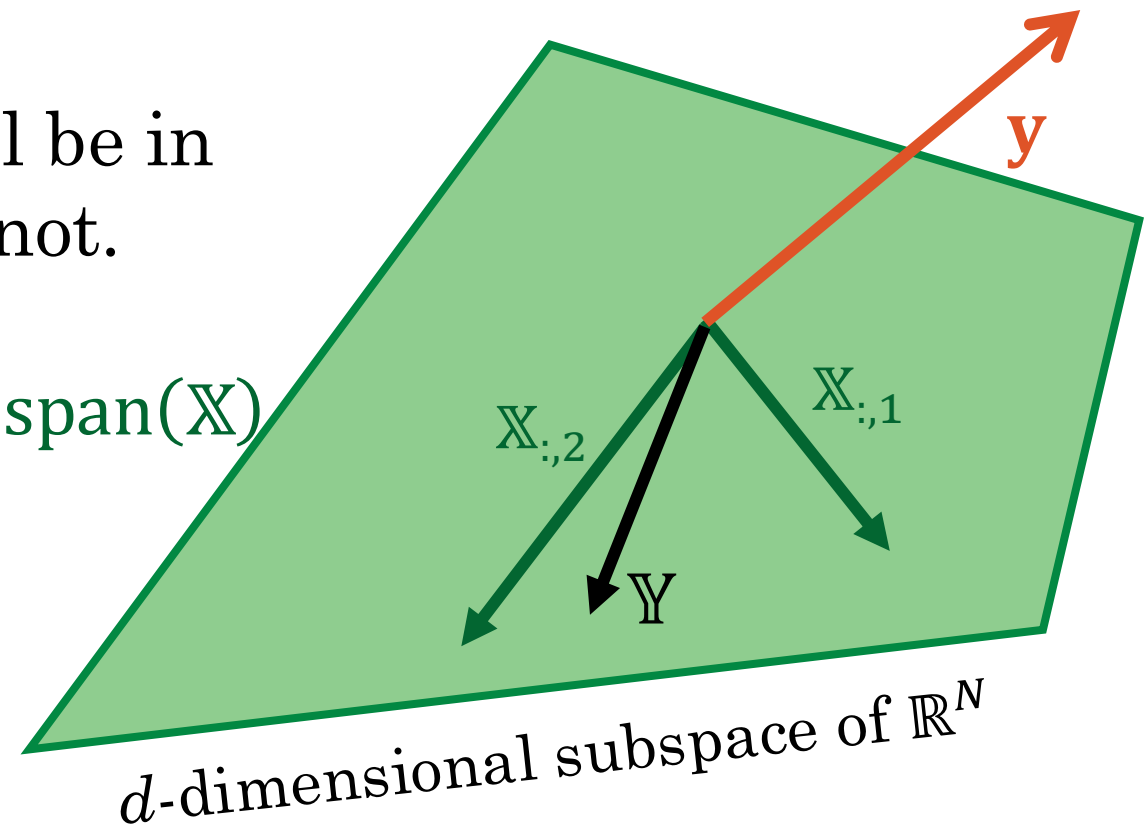
Prediction Is a Linear Combination of Columns

Our prediction of $\mathbb{Y} = \mathbb{X} \vec{w} = \sum_j w_j \mathbb{X}_{:,j}$ is a linear combination of columns of $\mathbb{X}$.

$$Y = Aw = Xw$$

Interpret: Our linear prediction $\mathbb{Y}$ will be in $\text{span}(\mathbb{X})$, even if observed values $\mathbf{y}$ are not.

Goal: Find vector of $\mathbb{Y}$ (predictions) in $\text{span}(\mathbb{X})$ that is closest to $\mathbf{y}$.

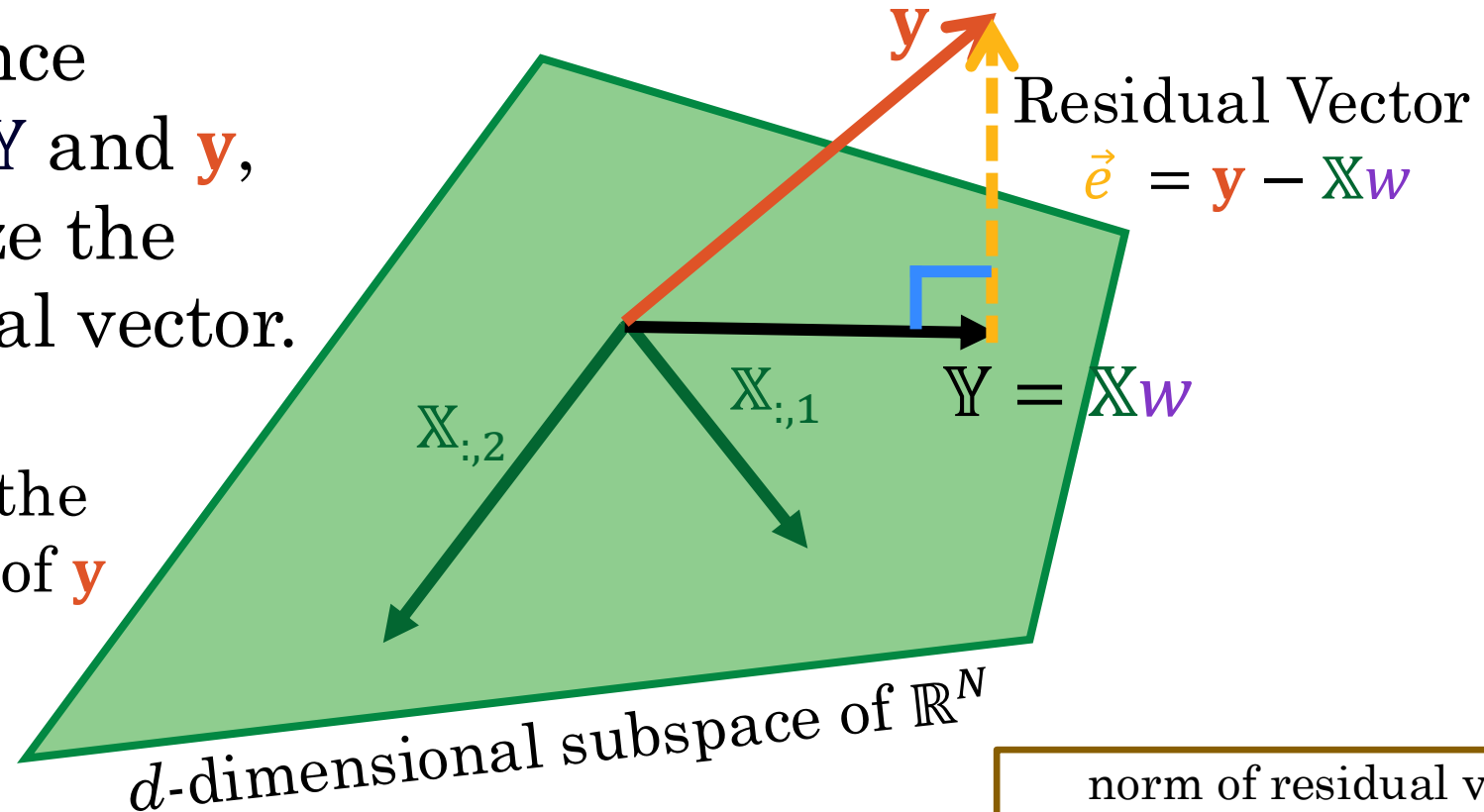


Finding Optimum Predictions ($\mathbb{Y}$)

$$Y = A w = \mathbb{X} w$$

To minimize distance between vector of $\mathbb{Y}$ and $\mathbf{y}$, we should minimize the norm of the residual vector.

$\vec{e}$ is minimized if it is the orthogonal projection of $\mathbf{y}$ on the $\text{span}(\mathbb{X})$.



norm of residual vector

$$\|\vec{e}\|_2 = \sqrt{\sum_{d=0}^D e_d^2}$$

Finding Optimum Model parameters ($\mathbf{w}$)

Vector $\vec{e}$ is orthogonal to the $\text{span}(\mathbb{X})$, meaning that:

$$\mathbb{X}^T \vec{e} = 0$$

$$\mathbb{X}^T (\mathbf{y} - \mathbb{Y}) = 0$$

$$\mathbb{X}^T \mathbf{y} - \mathbb{X}^T \mathbb{X} \mathbf{w}^* = 0$$

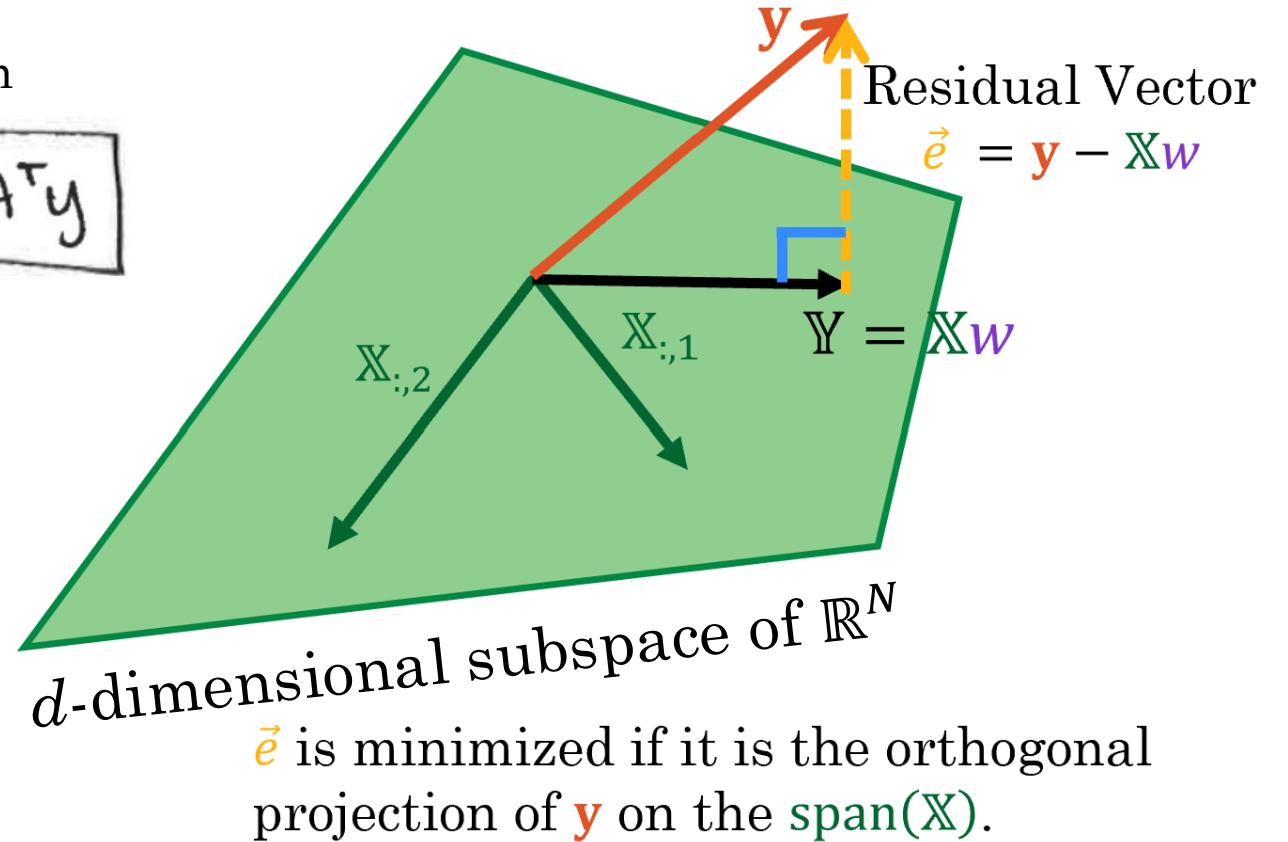
$$\mathbb{X}^T \mathbf{y} = \mathbb{X}^T \mathbb{X} \mathbf{w}^*$$

➡ $\mathbf{w}^* = (\mathbb{X}^T \mathbb{X})^{-1} \mathbb{X}^T \mathbf{y}$

If $\mathbb{X}^T \mathbb{X}$ is invertible

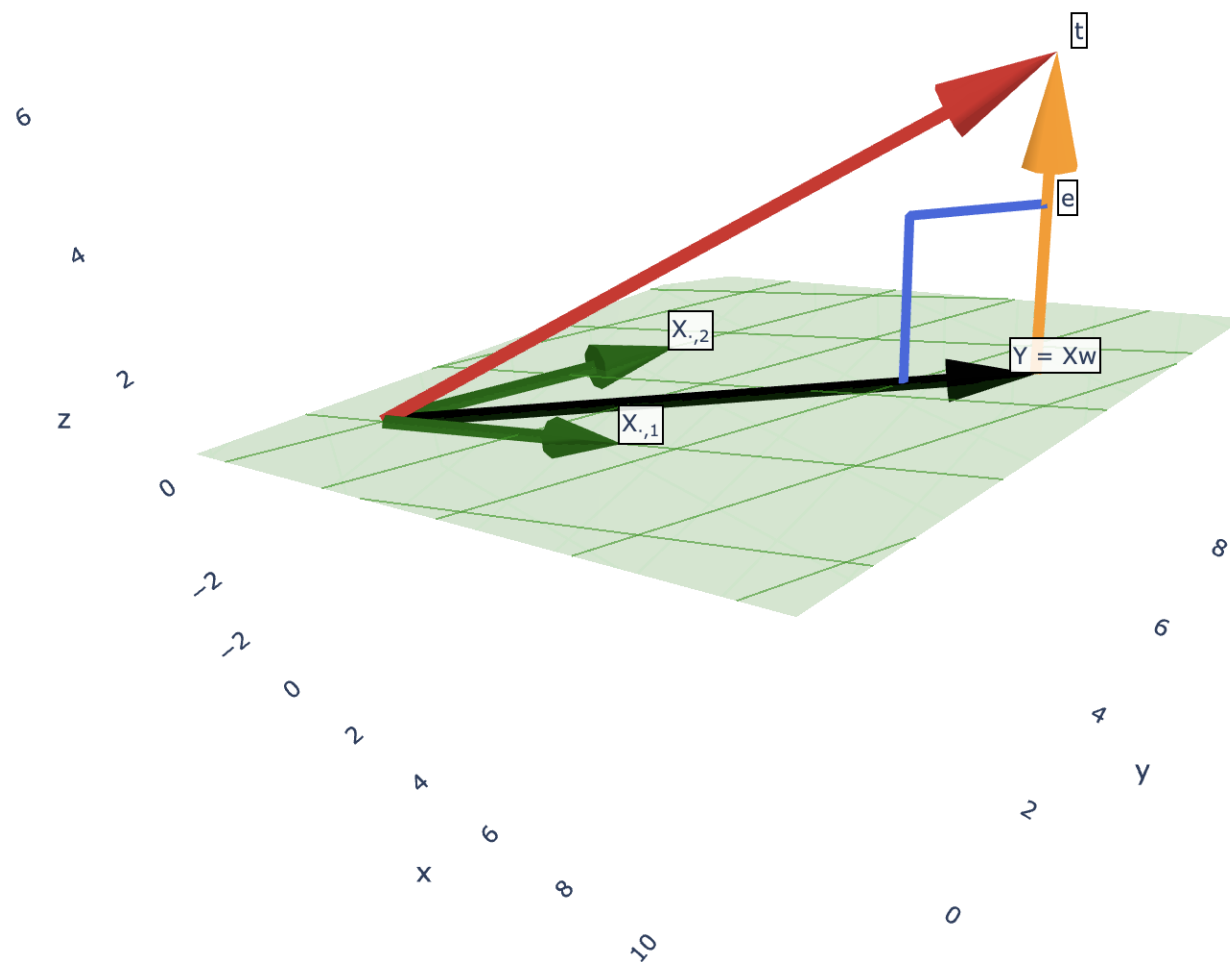
MLE solution

$$\mathbf{w} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{y}$$



Geometry of Least Squares in Plotly

[Interactive link](#)



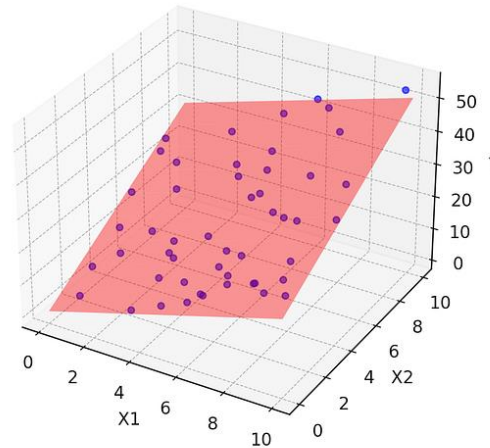
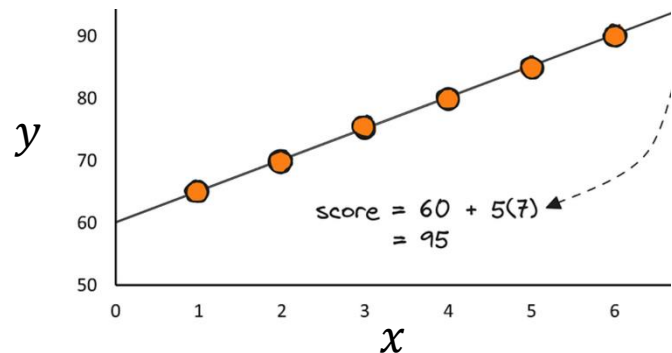
What can go wrong in linear regression

$$w^* = (\mathbb{X}^T \mathbb{X})^{-1} \mathbb{X}^T \mathbf{y} \quad w^* \in \mathbb{R}^{N \times d}$$

"left pseudo-inverse" of A
 $(A^T A)^{-1} A^T * A = I$

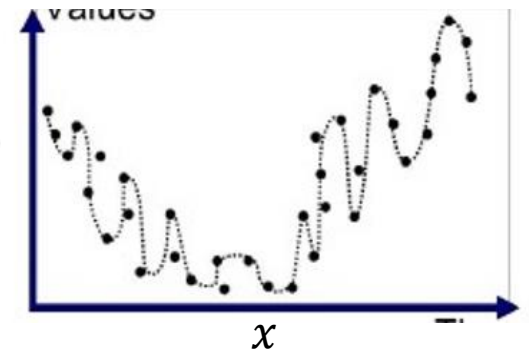
When is $\mathbb{X}^T \mathbb{X} \in \mathbb{R}^{d \times d}$ invertible (d features)? ($\mathbb{X} \in \mathbb{R}^{N \times d}$)

- When $\mathbb{X}^T \mathbb{X}$ has full rank (rank= d), aka when it is PD (positive definite, i.e., all eigenvalues are positive).
- Failure to be PD occurs when the d features in $\mathbb{X}$, for the given data $\{x_j\}$, are linearly dependent, then $\text{rank}(\mathbb{X}^T \mathbb{X}) < d$.
- e.g., when $d > N$ (then $\text{rank}(\mathbb{X}^T \mathbb{X}) \leq N < d$).
- What about when $d = N$?



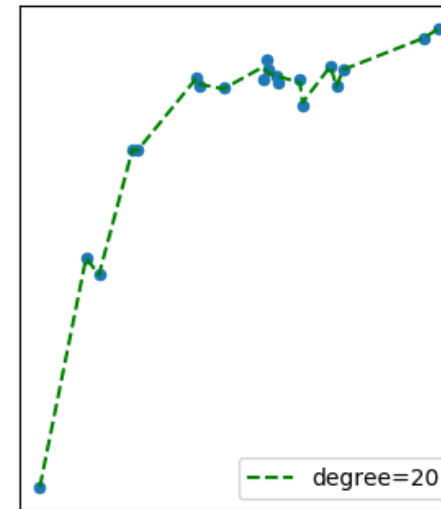
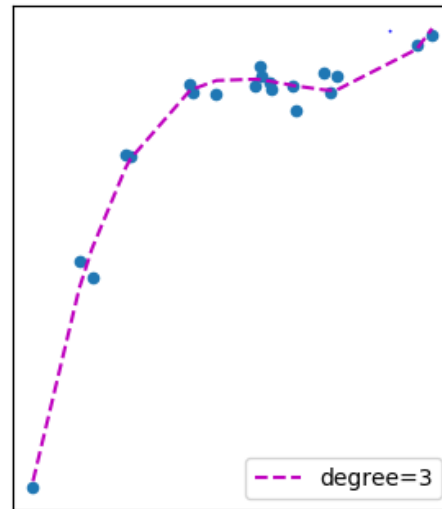
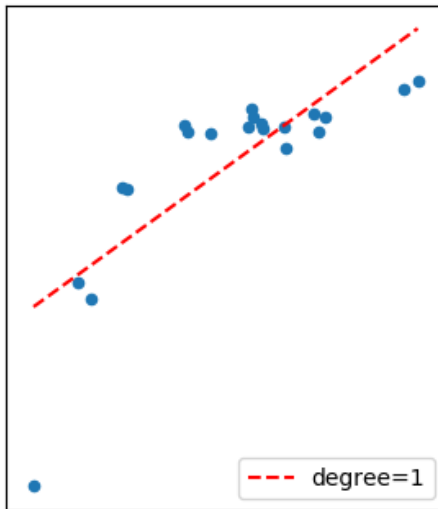
$$w = (\Phi(\mathbb{X})^T \Phi(\mathbb{X}))^{-1} \Phi(\mathbb{X})^T \mathbf{y}$$

$$y = w \phi(x)$$



What can go wrong in linear regression

- Even when we don't perfectly fit the training data, we are still in danger of overfitting (worse prediction on test set).
- Our goal is not to fit a line through the training data exactly, it is to do well on unseen test cases!



What can go wrong in linear regression

Two main categories of fixes:

1. Remove features until the problem is well-behaved (“feature selection”).
2. Leave the features as they are, but *add constraints to the system* to “tighten it up” (aka “regularization”)—next topic!

Roadmap

- Linear Regression
 - Finish Probabilistic/MLE view
 - Geometric view
 - Regularization

Intuition of why ∞ # of w_{MLE} solutions

- Suppose we have 2 *linearly dependent features* in the training data such that $\alpha x_1 = x_2$.
- Suppose we found one MLE solution, $\hat{w}$.
- Then for any training data point, $\hat{y} = x^T \hat{w}$.

Of all the $\{w_{MLE}\}$ with same error, is there one that intuitively might be generally be better?

$$\begin{aligned} &= [x_1 \ x_2] \begin{bmatrix} \hat{w}_1 \\ \hat{w}_2 \end{bmatrix} \\ &= \hat{w}_1 x_1 + \hat{w}_2 x_2 = \hat{w}_1 x_1 + \hat{w}_2 \alpha x_1 \\ &= (\hat{w}_1 + \hat{w}_2 \alpha) x_1 = (\hat{w}_1 + \hat{w}_2 \alpha + \beta - \beta) x_1 \text{ for any } \beta. \\ &= ((\hat{w}_1 + \beta) + (\hat{w}_2 \alpha - \beta)) x_1 \text{ for any } \beta. \\ &= (\hat{w}_1 + \beta) x_1 + (\hat{w}_2 \alpha - \beta) \frac{x_2}{\alpha} = (\hat{w}_1 + \beta) x_1 + (\hat{w}_2 \alpha - \beta) \frac{1}{\alpha} x_2 \\ &= [x_1 \ x_2] \begin{bmatrix} \hat{w}_1 + \beta \\ \frac{\hat{w}_2 \alpha - \beta}{\alpha} \end{bmatrix} = x^T \tilde{w} = \hat{y} \text{ for } \tilde{w} = \begin{bmatrix} \hat{w}_1 + \beta \\ (\hat{w}_2 \alpha - \beta) / \alpha \end{bmatrix} \end{aligned}$$

Intuition for choosing one specific $\hat{\mathbf{w}}$.

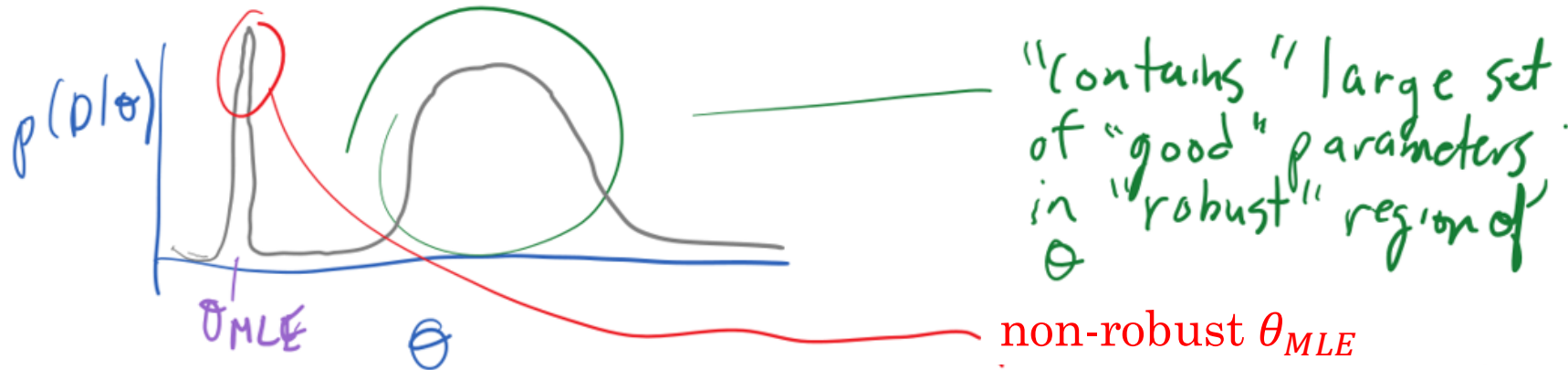
Of the ∞ solutions for $\mathbf{w}_{MLE}$, choose the one with the least norm, $\|\mathbf{w}_{MLE}\|_2$.
Why might this be a good idea? $\hat{\mathbf{y}} = \mathbf{x}^T \hat{\mathbf{w}}$

- We would like each feature to have as little effect on the outcome as possible while still predicting well, so that model behaves “gracefully”.
- Consider prediction, $\hat{\mathbf{y}} = \mathbf{w}^T \mathbf{x}$. How much does the prediction change when we perturb, $\mathbf{x}' = \mathbf{x} + \delta$, for different norm $\mathbf{w}$?
- With smaller coefficients the model is less sensitive to noisy data.
- What about for non-degenerate linear regression ($\mathbf{A}^T \mathbf{A}$ is invertible)?
- Yes! For many problems (and models), small param norm is a good idea.
- Small norm is one *e.g.*, of *regularization*:
 - in effect, reduce # free parameters, while keeping the same set of parameters!

Recall how MLE can go wrong?

MLE yields a "point estimate" of our parameter

- When we perform MLE, we get just one single estimate of the parameter, θ , rather than a distribution over it which captures uncertainty.
- In Bayesian statistics, we obtain a (posterior) distribution over θ . We will touch more on this in a few lectures.



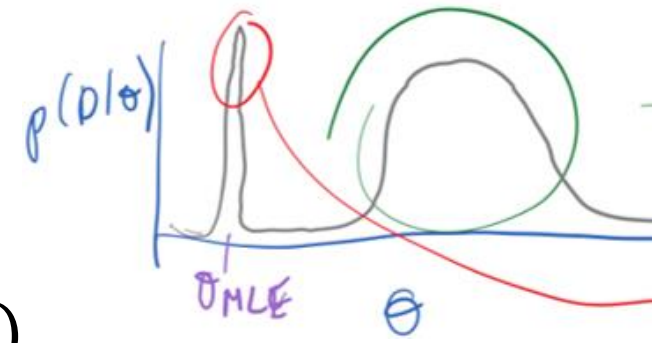
L2 regularized linear regression

To *shrink* $\mathbf{w}$ to have smaller norm than the MLE solution, we add a “penalty” term to the loss function:

$$\mathcal{L} = (\mathbf{y} - A\mathbf{w})^T (\mathbf{y} - A\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2$$
$$\mathbf{w}_{L_2} = \operatorname{argmin}_{\mathbf{w}} (\mathbf{y} - A\mathbf{w})^T (\mathbf{y} - A\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2$$

Also called “Ridge” regression, or L2 linear regression.
Related to *Bayesian* modeling (next).

The Bayesian modelling approach



- Bayesians put a *prior distribution* on the parameters, $p(\theta)$.
- Then they seek to compute the *posterior distribution*, $p(\theta|D)$.
- Then, predictive distribution is given by

$$p(y|x) = \int_{\theta} p(y|x, \theta) p(\theta|D) d\theta = E_{\theta}[p(y|x, \theta)]$$

- Procedurally, this is done using Bayes' rule:

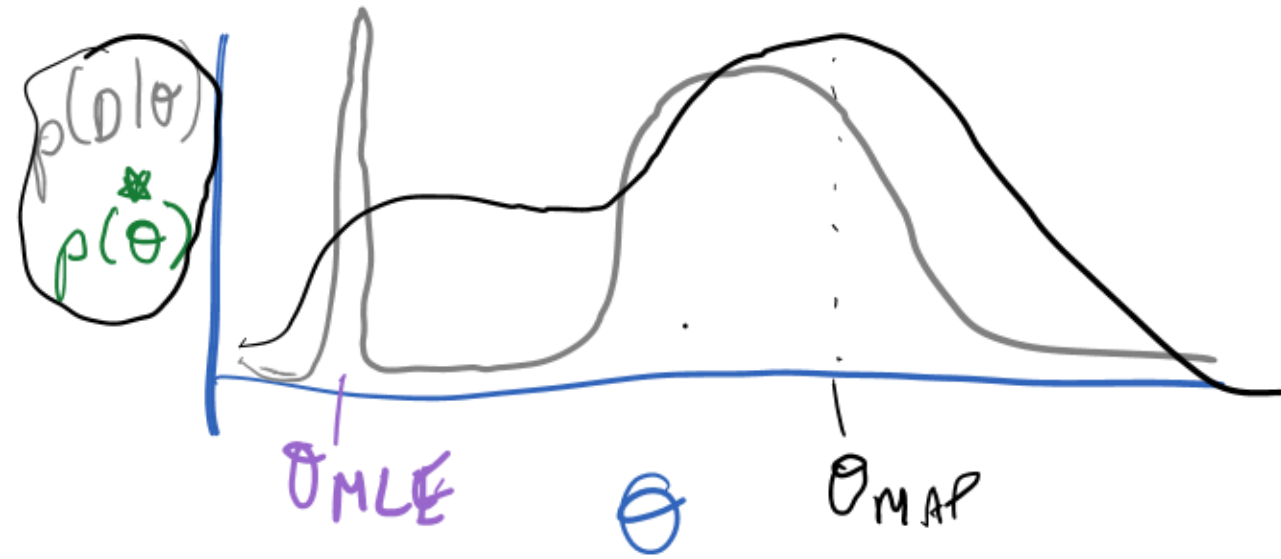
$$p(\theta|D) = \frac{p(D|\theta)p(\theta)}{p(D)}.$$

- Difficult in practice! $p(D) = \int_{\theta} p(D, \theta) d\theta = \int_{\theta} p(D|\theta)p(\theta) d\theta$
- We will be lazy, instead being fake Bayesians, yielding L2 regression:
- $\theta_{lazy} = \operatorname{argmax}_{\theta} p(\theta|D)$ *Maximum A Posteriori (MAP) estimation.*

MAP: the lazy Bayesian (*Maximum A Posteriori*)

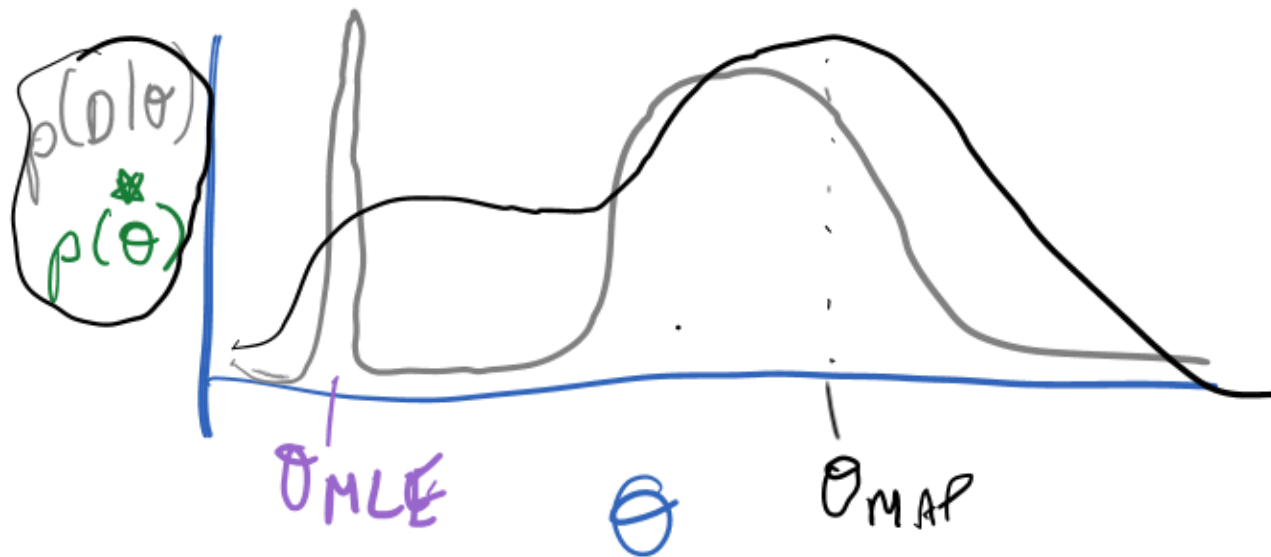
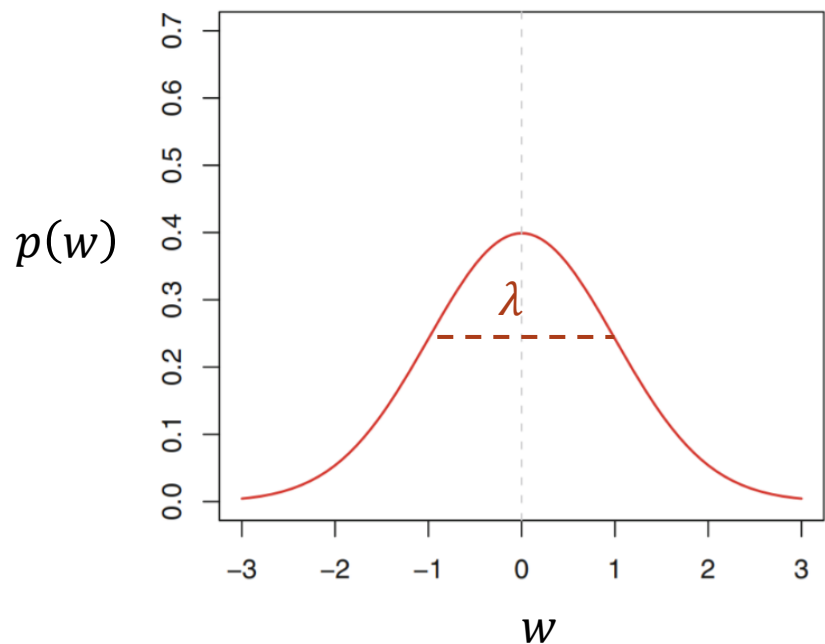
- Still use a prior over parameters, $p(\theta)$.
- Finds point estimate of the parameter that maximizes the posterior.
- $\theta_{MAP} = \operatorname{argmax}_{\theta} p(\theta|D)$

$$\begin{aligned} &= \operatorname{argmax}_{\theta} \frac{p(D|\theta) \cancel{p(D)}}{\cancel{p(D)}} \\ &= \operatorname{argmax}_{\theta} p(D|\theta) p(\theta) \end{aligned}$$



A prior for small weights yields L2 regression!

- Zero-mean prior, $p(w) = N(w; 0, \lambda I)$.
- Bayesian posterior, $p(w|D) = \frac{p(D|w)p(w)}{p(D)}$ is then “nice” in that everything is Gaussian (can analytically work it out using MVGs).



MAP for linear regression w Gaussian prior

$$\begin{aligned}w_{MAP} &= \operatorname{argmax}_w \log p(D|w) p(w) = \operatorname{argmax}_w \log p(D|w) + \log N(w; 0, \lambda I) \\&= \operatorname{argmax}_w \sum_{i=1}^N \log N(y_i | w^T x_i, \sigma^2) + \log N(w | 0, \lambda I) \\&= \operatorname{argmin}_w + \frac{1}{2\sigma^2} (y - Aw)^T (y - Aw) - \sum_{i=1}^d \log \left[\frac{1}{\sqrt{2\pi\lambda}} \exp \left(-\frac{(w_i - 0)^2}{2\lambda} \right) \right] \\&= \operatorname{argmin}_w \frac{1}{2\sigma^2} (y - Aw)^T (y - Aw) + \sum_{i=1}^d \left[-\log \frac{1}{\sqrt{2\pi\lambda}} + \frac{w_i^2}{2\lambda} \right] \\&= \operatorname{argmin}_w \frac{1}{2\sigma^2} (y - Aw)^T (y - Aw) + \sum_d \frac{1}{2\lambda} w_i^2 \\&= \operatorname{argmin}_w \frac{1}{2\sigma^2} (y - Aw)^T (y - Aw) + \frac{1}{2\lambda} \|w\|_2^2 \\&= \operatorname{argmin}_w (y - Aw)^T (y - Aw) + \cancel{2\sigma^2} \frac{1}{2\lambda} \|w\|_2^2 \\&= \operatorname{argmin}_w (y - Aw)^T (y - Aw) + \lambda' \|w\|_2^2 \quad \text{for } \lambda' = \frac{\sigma^2}{\lambda}.\end{aligned}$$

Equivalence between
MAP w Gaussian prior
and L2 regression!

Obtaining the MAP/L2 solution

$$\begin{aligned}w_{L_2} &= \operatorname{argmin}_w (y - Aw)^T (y - Aw) + \lambda \|w\|_2^2 \\&= \operatorname{argmin}_w (y - Aw)^T (y - Aw) + \lambda w^T w\end{aligned}$$

Take partial derivative and set to zero:

$$\nabla_w \mathcal{L}_{MAP} = -2A^T y + 2A^T A w + 2\lambda I w$$

$$\rightarrow 0 = -A^T y + A^T A w + \lambda I w$$

$$\rightarrow A^T y = (A^T A + \lambda I) w$$

$$\rightarrow (A^T A + \lambda I)^{-1} A^T y = w$$

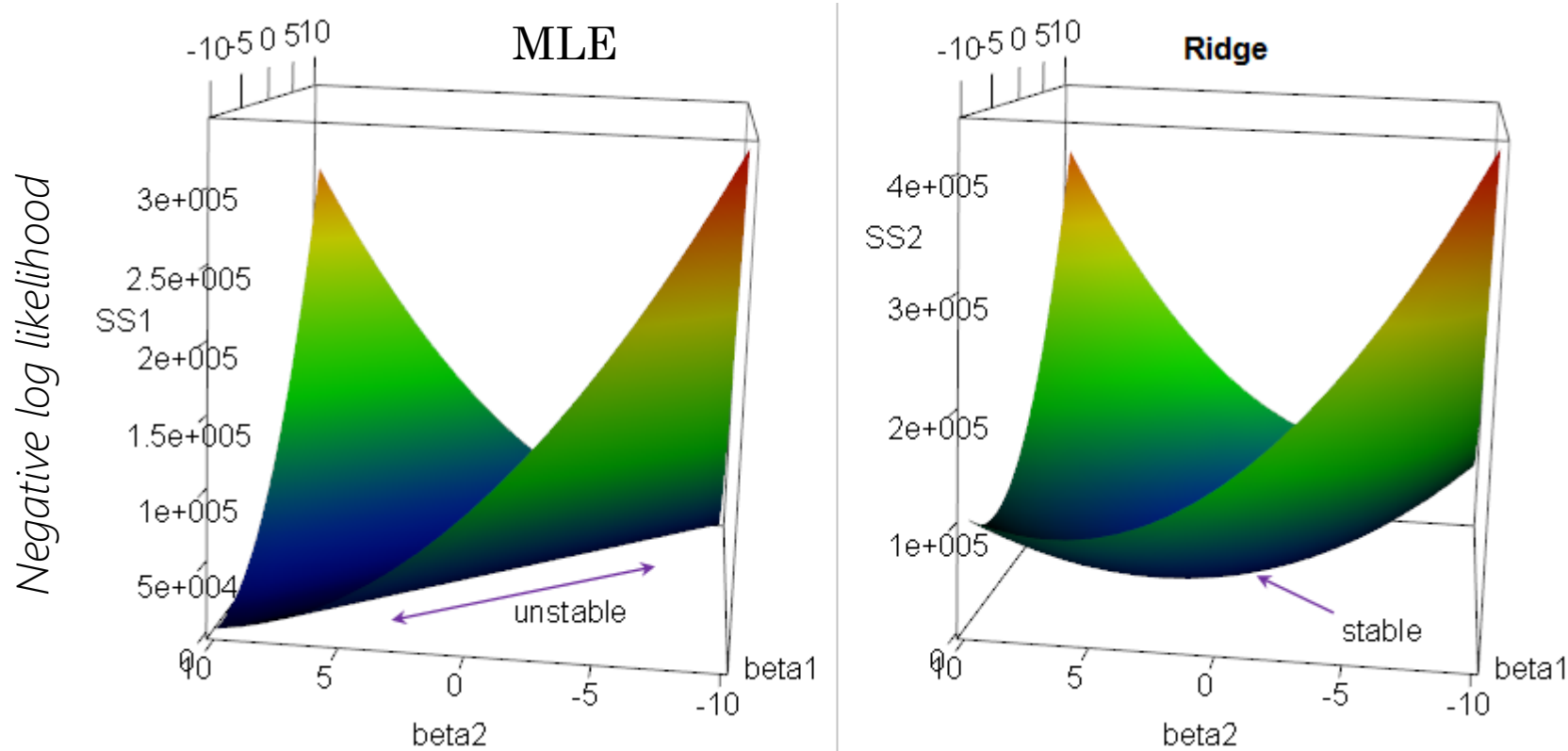
$$\text{So } w_{L_2} = (A^T A + \lambda I)^{-1} A^T y.$$

If $\lambda > 0$, we can invert $(A^T A + \lambda I)$.

$$\begin{aligned}A &= \Phi D \Phi^T \\A^{-1} &= \Phi D^{-1} \Phi^T \\ \text{where } D^{-1} &= \begin{bmatrix} \lambda_1^{-1} & & \\ & \lambda_2^{-1} & \\ & & \ddots \\ & & & \lambda_n^{-1} \end{bmatrix}\end{aligned}$$

Bending the “flat ridge” to get one optimal w

When some features are linearly dependent (can't invert $A^T A$), we have ∞ many equally good solutions that form a ridge.



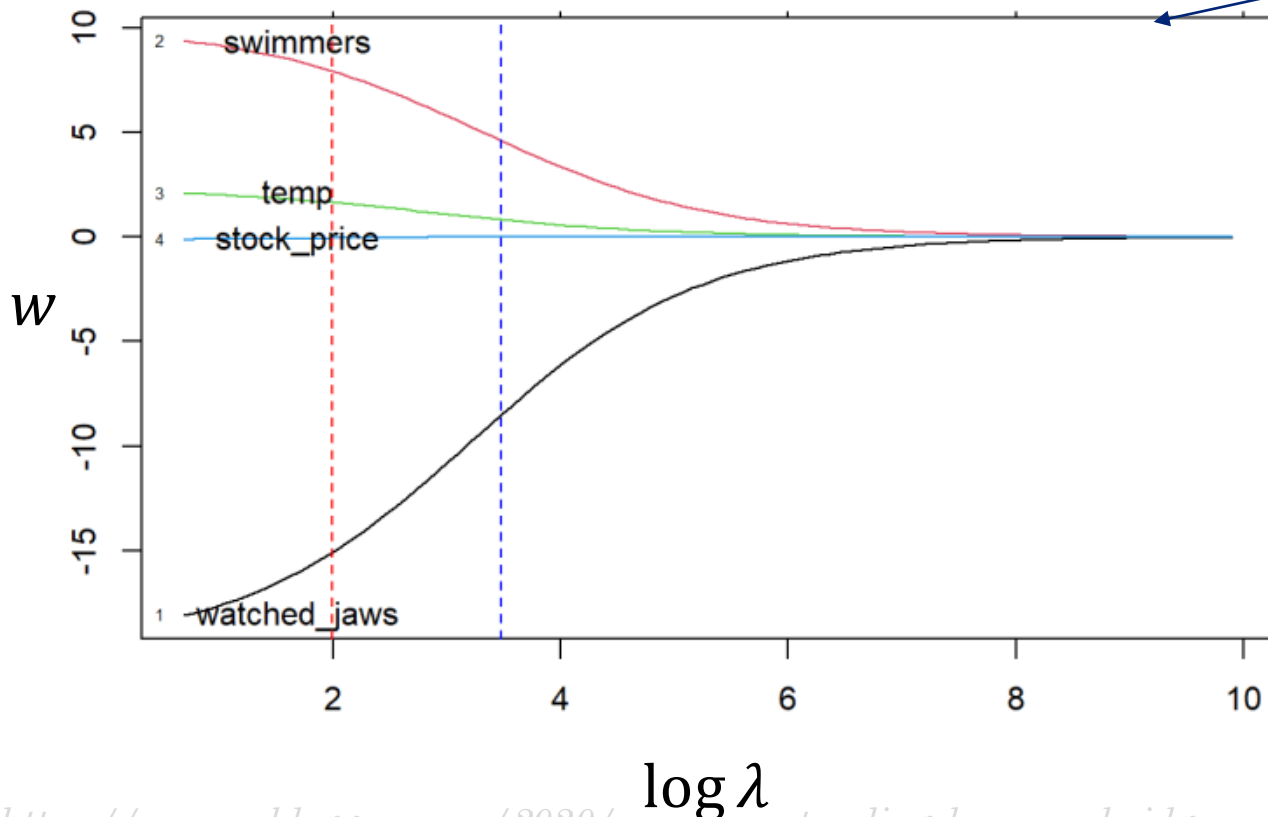
$$w_{L_2} = (A^T A + 0)^{-1} A^T y.$$

$$w_{L_2} = (A^T A + \lambda I)^{-1} A^T y.$$

Effect of value of λ

$$\mathcal{L}_{MAP} = (y - Aw)^T (y - Aw) + \lambda \|w\|_2^2$$

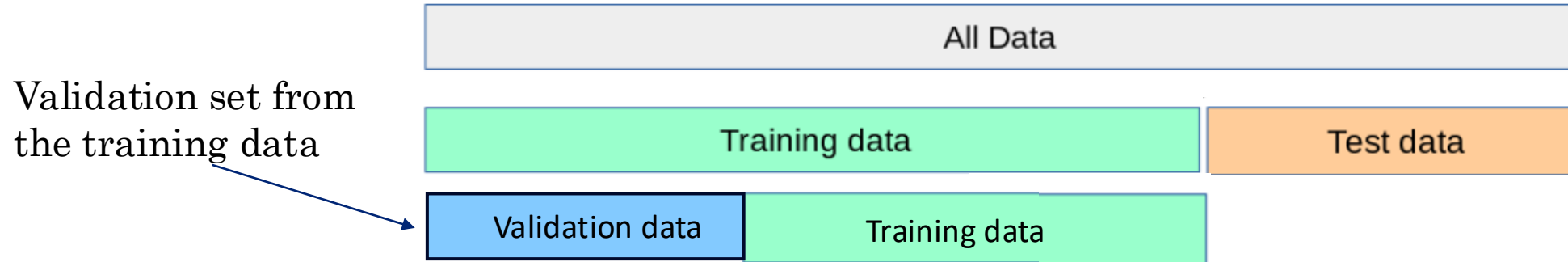
of non-zero features



- Practically, how should we set λ ?
- Can we treat it as a parameter in the loss, and minimize wrt it?
- No: cannot use MLE!
- Need independent data, a *validation set* on which to evaluate the loss.

Train/validation/test split

$$\mathcal{L}_{MAP} = (y - Aw)^T (y - Aw) + \lambda \|w\|_2^2$$



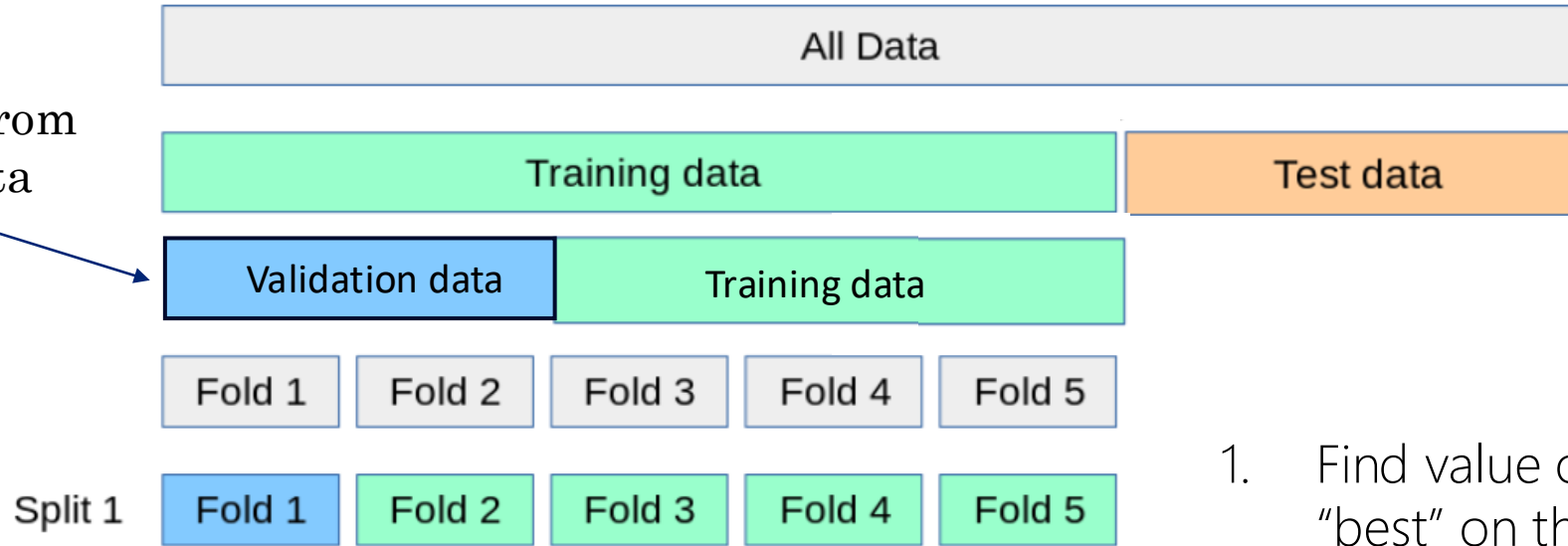
1. Find value of hyperparameter that is "best" on the **validation set**.
2. Quantify performance on the **test data**.

"Best"/quantify require computing e.g., log likelihood on the held out data, and this is our metric.

Train/validation/test split

$$\mathcal{L}_{MAP} = (y - Aw)^T (y - Aw) + \lambda \|w\|_2^2$$

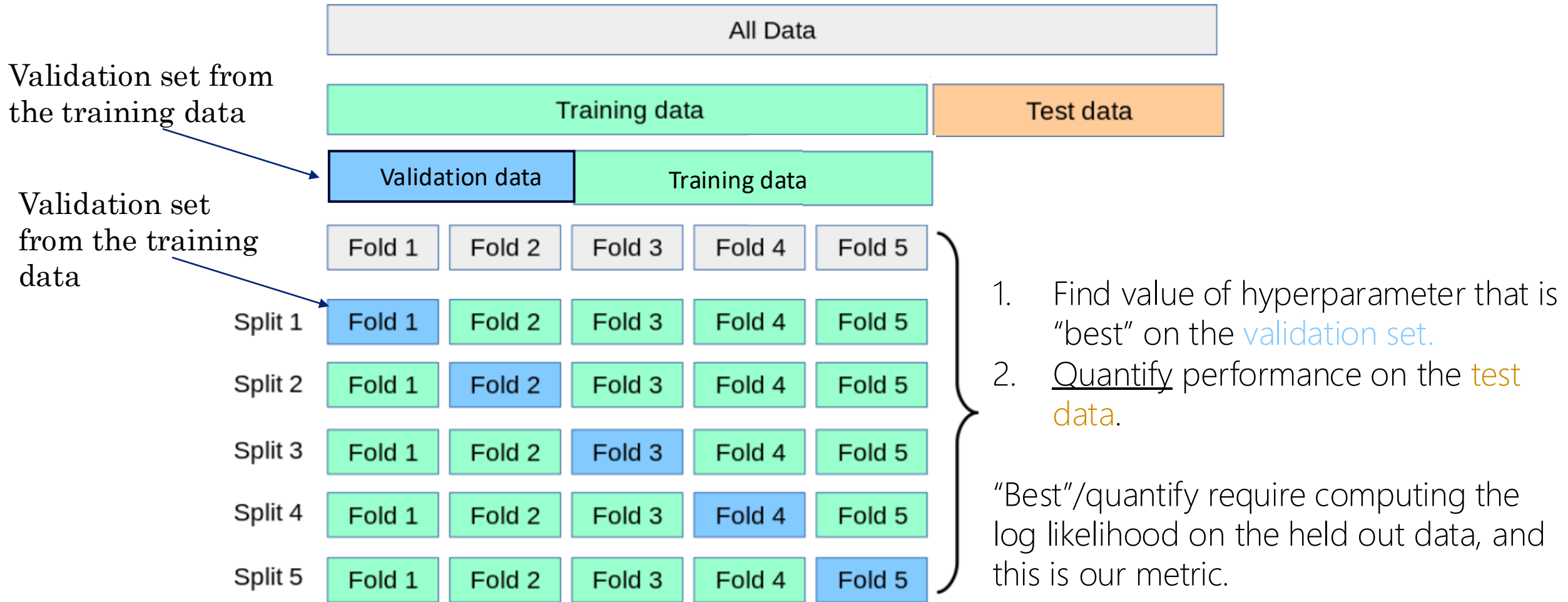
Validation set from
the training data



1. Find value of hyperparameter that is "best" on the **validation set**.
2. Quantify performance on the **test data**.

"Best"/quantify require computing the log likelihood on the held out data, and this is our metric.

K-fold cross-validation

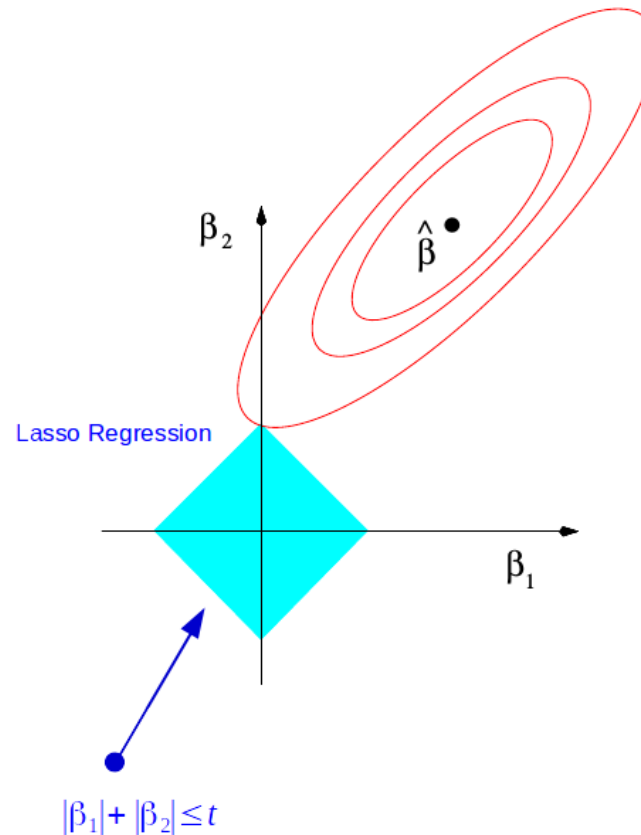


Other priors for MAP in linear regression?

- What if wanted to have the linear function only depend on a few features (*i.e.*, the coefficients of $\mathbf{w}$ are sparse)?
- Add a penalty that counts the # of non-zero weights—an L_0 penalty, $\lambda \|\mathbf{w}\|_0$.
- Not differentiable! (becomes combinatorial optimization—nasty).
- But...the L_1 norm penalty, $\lambda \|\mathbf{w}\|_1 = \lambda \sum_d |w_d|$, tends to induce sparse $\mathbf{w}$ —differentiable everywhere except at $\mathbf{w}_i = \mathbf{0}$ which can be worked around.
- This is called *Lasso* regression, or L_1 -penalized regression.

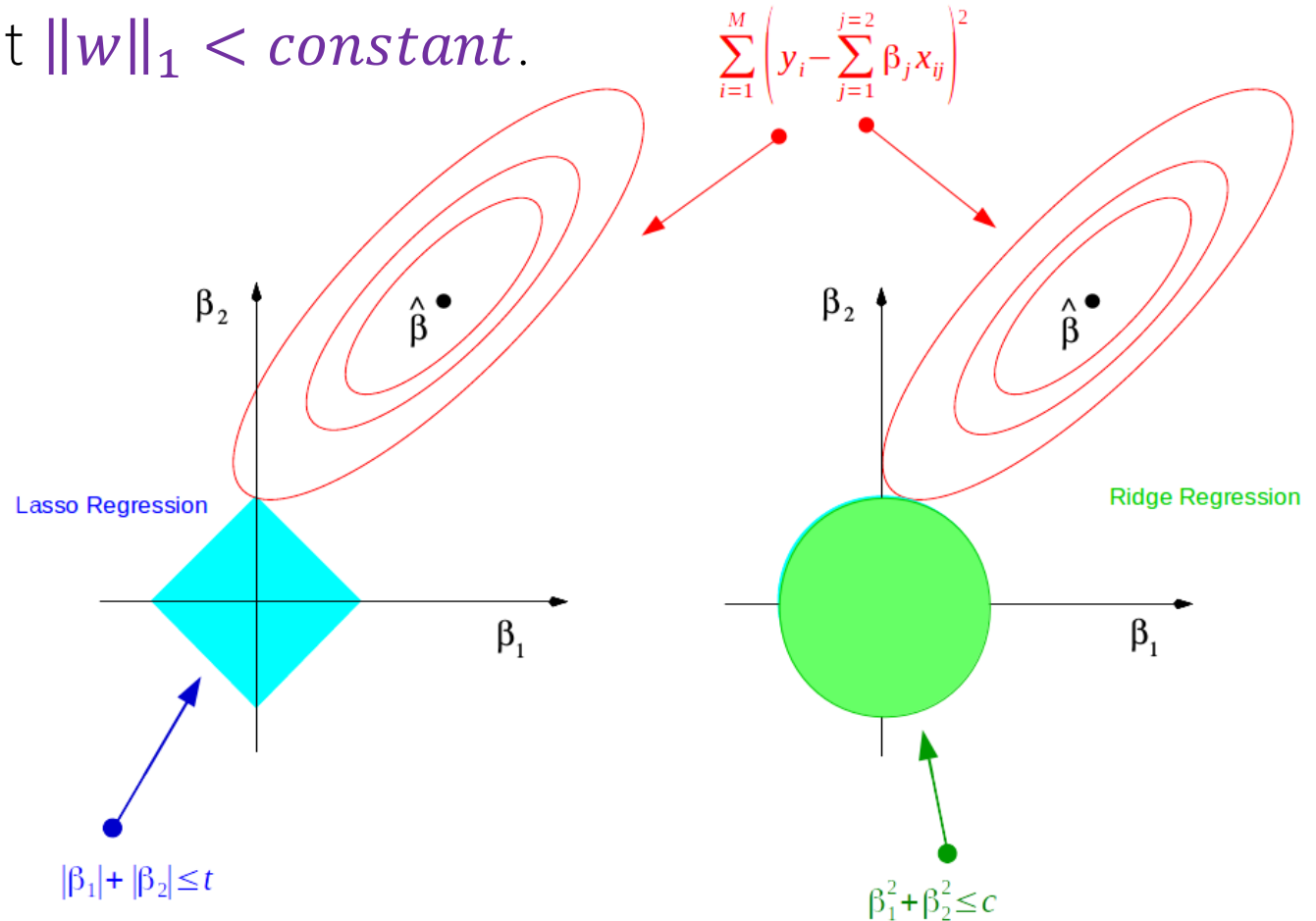
L_1 -penalized linear regression, aka *Lasso*

- $w_{L_1} = \operatorname{argmin}_w (y - Aw)^T (y - Aw) + \lambda \|w\|_1$
- Why does the L_1 norm penalty tends to induce sparse w ?
- Equivalent to MLE with constraint $\|w\|_1 < \text{constant}$.
- “Pointy” constraint surface is jutting out along the axes.
- In many cases, the L_1 norm constraint will cause the unconstrained solution to intersect the constraint at a corner.
- The corners are where some coefficients are 0, which is a sparse solution..

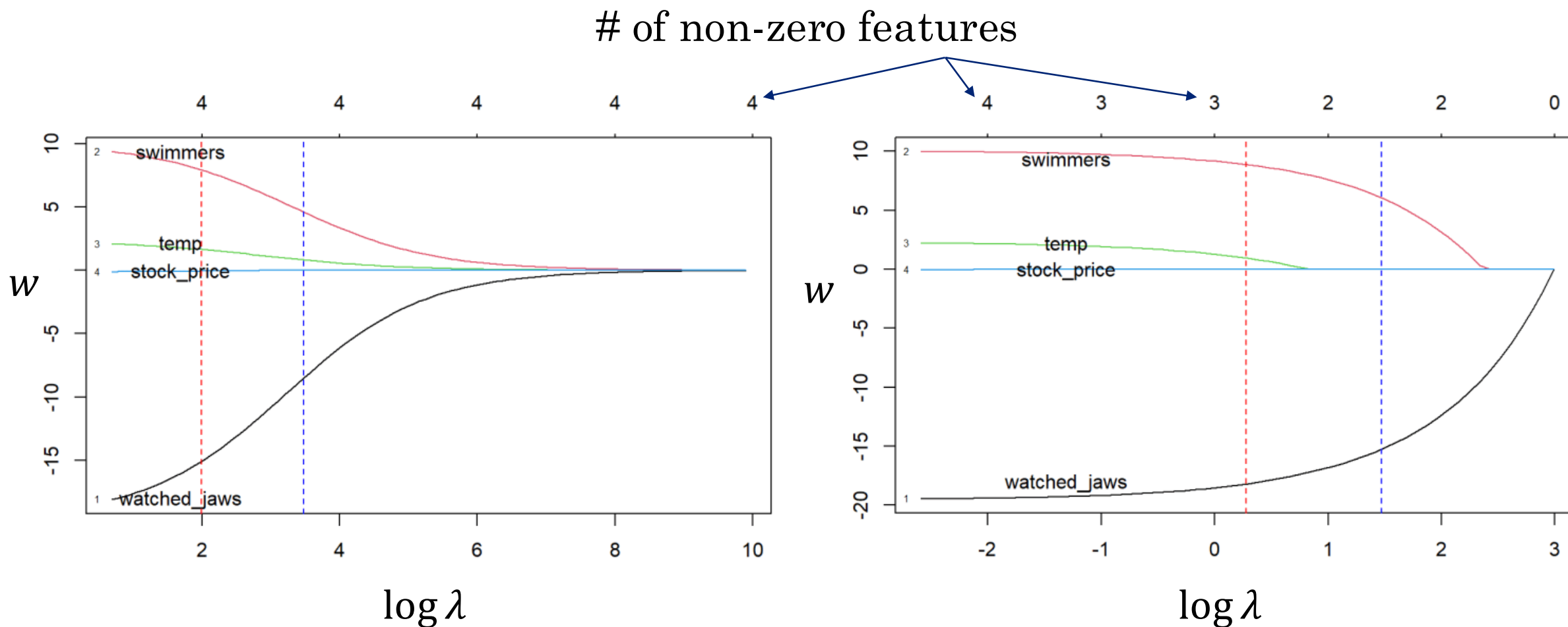


L_1 -penalized linear regression, aka *Lasso*

- $w_{L_1} = \operatorname{argmin}_w (y - Aw)^T (y - Aw) + \lambda \|w\|_1$
- Why does the L_1 norm penalty tends to induce sparse w ?
- Equivalent to MLE with constraint $\|w\|_1 < \text{constant}$.
- "Pointy" constraint surface is jutting out along the axes.
- In many cases, the L_1 norm constraint will cause the unconstrained solution to intersect the constraint at a corner.
- The corners are where some coefficients are 0, which is a sparse solution..



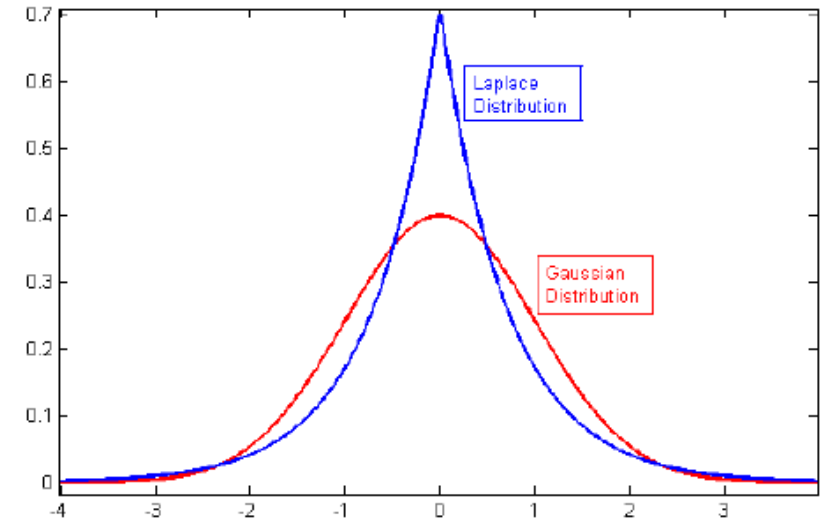
Ridge vs Lasso: shrinkage vs sparsity



MAP interpretation for Lasso/ L_1 -penalized linear regression?

- L_2 regression arose from a $N(0, \lambda I)$ prior.
- Is there a prior corresponding to L_1 ?
- Yes, the Laplace prior!

$$p(w) = \frac{1}{2\lambda'} \exp\left(-\frac{\|w\|_1}{\lambda'}\right).$$

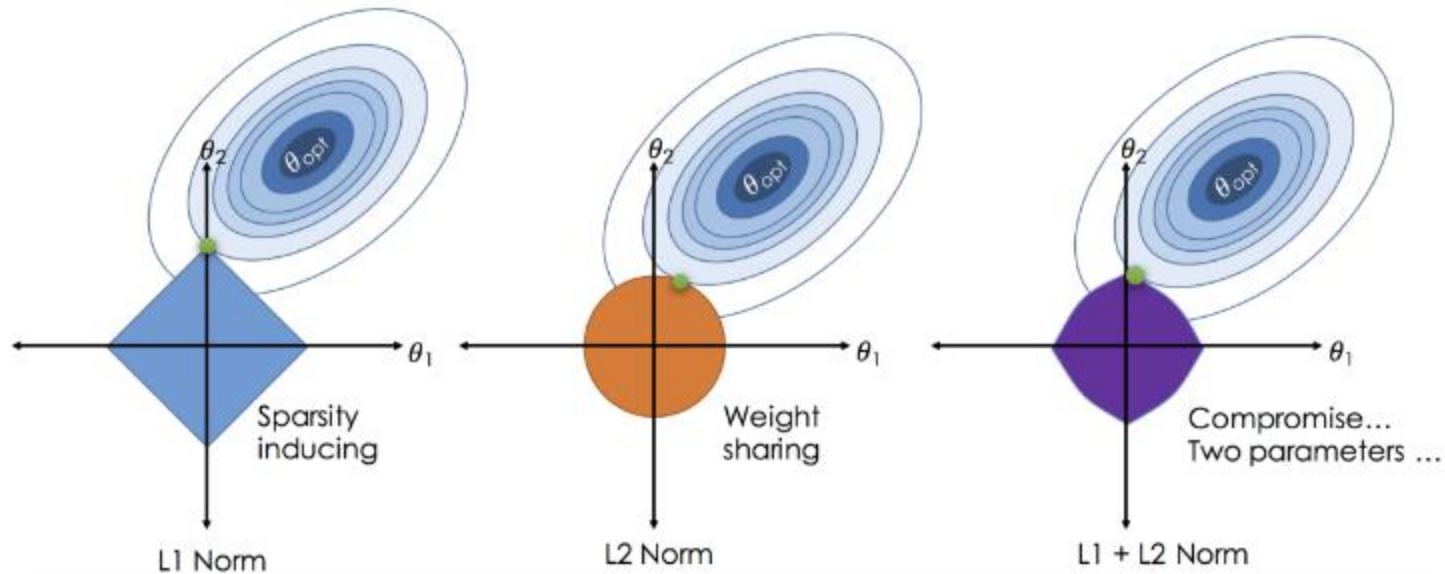


Issues with LASSO:

- If highly correlated features, tends to ignore all but one.

Combine L_1 and L_2 penalties?

Yes, "elastic net regression".



Extra slides (not responsible for)

Real application: Genome-Wide Association Studies (GWAS)

Input:

- A set of people with/without a disease
- Measure a large set of genetic markers for each person (e.g., SNPs: *single nucleotide polymorphism*).

Desired output:

- A list of genetic markers underlying the disease.

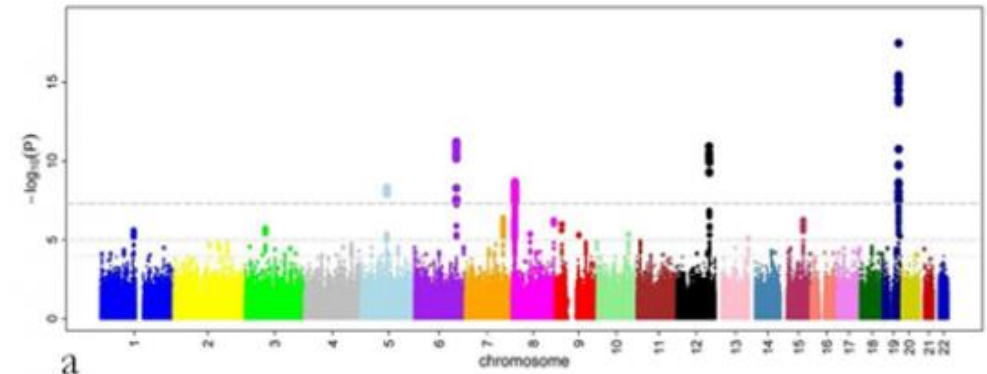


DNA from set of individuals

+
ATGTTGAATCTG
AAAGTGAAATGT
TATTATACGAAG
AAGTATTTGCTA
GACCTCAAACCT
CTTCATCATAAC



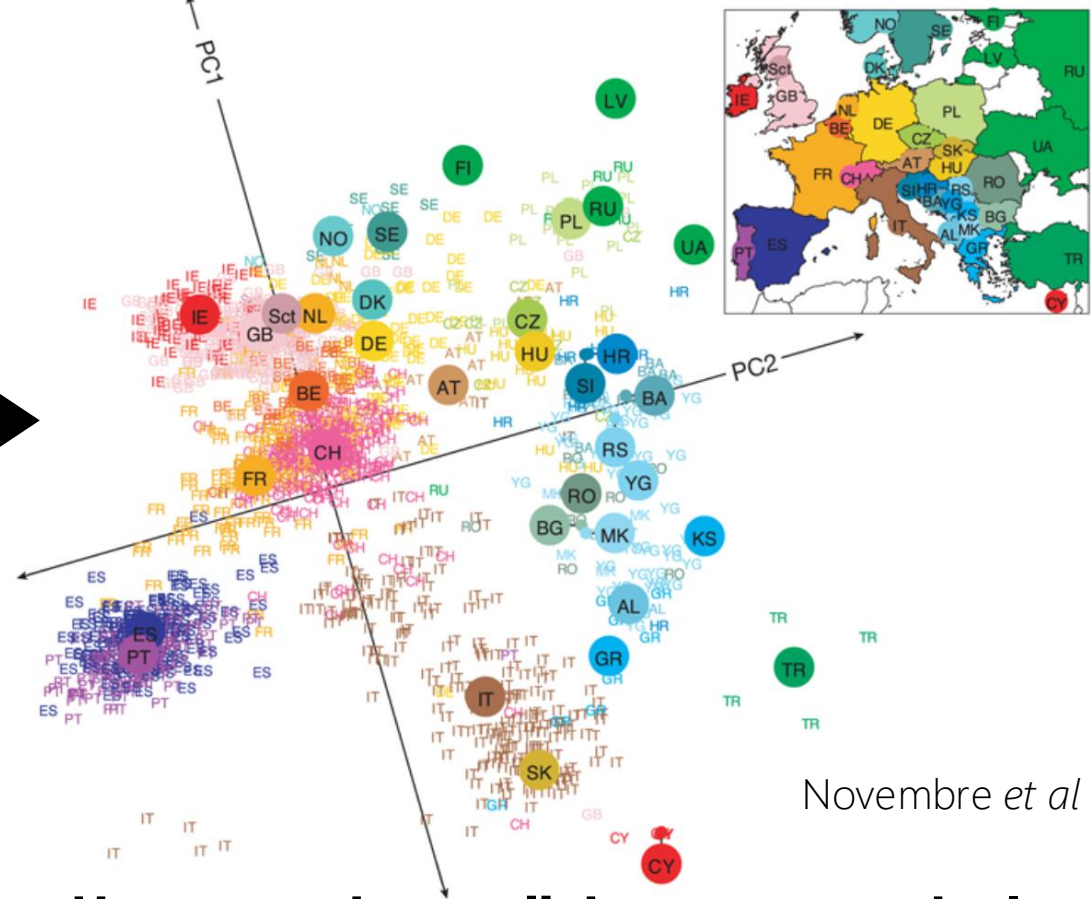
list of genetic markers



Spurious “signal” in the genetic markers owing to racial/pedigree confounders.



$N \times N$ covariance
matrix



Novembre *et al* Nature 2008

By including these “hidden dimensions” in a model, we can correct the problem.

Mixed Model Approach (GPR)

Generative model:

- sample a "latent ancestry vector"
- add as covariate

$$p(\vec{u}) = N(\vec{0}, \text{red square})$$

$$p(\vec{y}|\vec{u}) = N(X\beta + \eta\vec{u}, I\sigma^2)$$

"amount of
confounding
genetic signal"

➡ LIKELIHOOD

$$p(\vec{y}) = \int p(\vec{y}|\vec{u})p(\vec{u})d\vec{u}$$
$$p(\vec{y}) = N(X\beta, \eta^2 \text{red square} + I\sigma^2)$$

2. Exploit rank when few SNPs are used

If $\mathbf{K}$ is computed in a particular way, and $n > s_c$ then computations are linear in n .

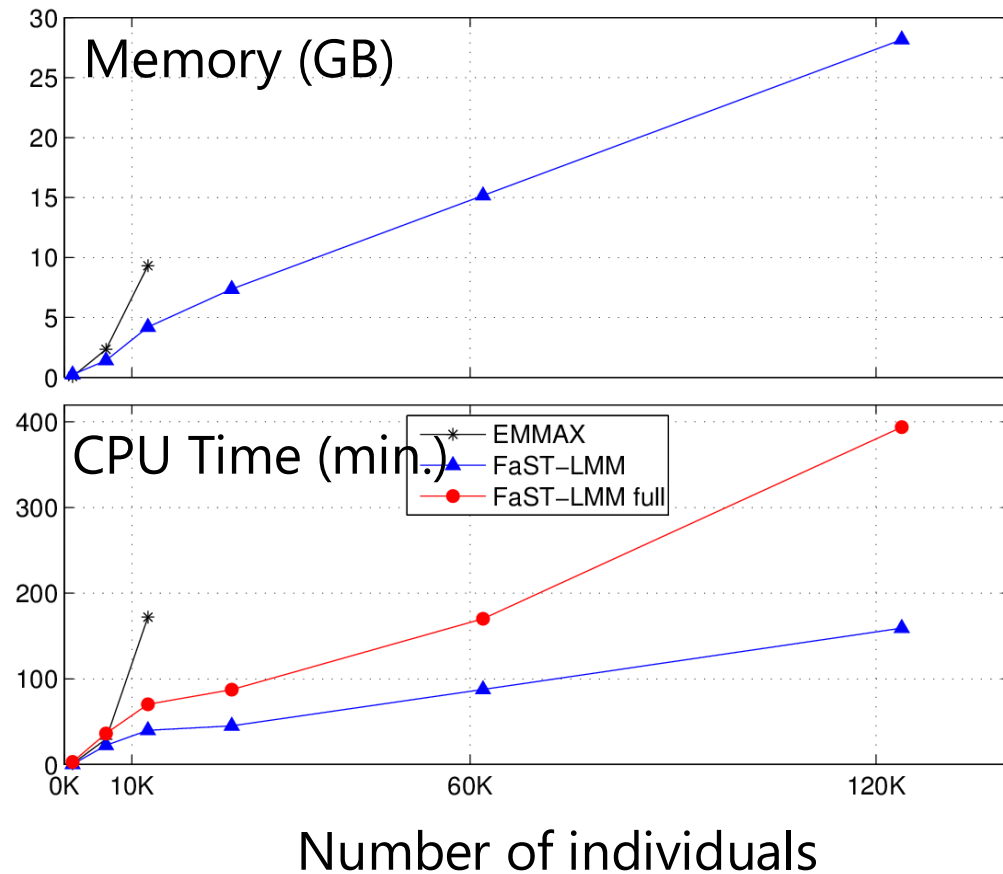
$$\begin{matrix} & \overbrace{\hspace{1cm}}^n \\ \underbrace{\hspace{1cm}}_n & \mathbf{K} \end{matrix} = \frac{1}{s_c} \begin{matrix} & \overbrace{\hspace{1cm}}^{s_c} \\ \underbrace{\hspace{1cm}}_n & \mathbf{W} \end{matrix} \times \mathbf{W}^T$$

e.g., the realized relationship matrix (RRM) [Visscher et al]

n is # people

s_c is # of SNPs used to compute $\mathbf{K}$

Experimental running time and memory



- **EMMAX** out of memory after 13K individuals
- **FaST-LMM** fixed variance components
- **FaST-LMM full** re-estimate variance components

Related papers:

Improved linear mixed models for genome-wide association studies

Jennifer Listgarten, C. Lippert, C. Kadie, R. Davidson, E. Eskin and D. Heckerman

(equal contributions and corresponding)

Nature Methods 2012, [doi:10.1038/nmeth.2037](https://doi.org/10.1038/nmeth.2037)

FaST Linear Mixed Models for Genome-Wide Association Studies

C. Lippert, Jennifer Listgarten, Y. Liu, C. Kadie, R. Davidson and D. Heckerman

(equal contributions and corresponding)

Nature Methods Aug. 2011, [doi:10.1038/nmeth.1681](https://doi.org/10.1038/nmeth.1681),

Epigenome-wide association studies without the need for cell-type

composition, James Zou, C. Lippert, D. Heckerman, Martin Aryee, Jennifer Listgarten, *Nature Methods* 2014 ([journal link](#))

FaST-LMM-Select for addressing confounding from spatial structure and rare

variants, Jennifer Listgarten, Christoph Lippert, David

Heckerman (equal contributions and corresponding)

Nature Genetics 2013 ([journal link](#))

A thought experiment

Consider:

- Use MLE on data, $D = \{x_i, y_i\}$ to get $\hat{y}_i = p_{\theta}(y|\Phi(x))$ using linear regression.
- Assume an abundance of data (millions of data points), and only 100 parameters.
- Suppose get accuracy $\mp \$1000$ of sale price when applying to held out part of our data.
- Can we assume this model will get $\mp \$1000$ on any test set that may come in the future?

Actual vs. predicted sale price of house

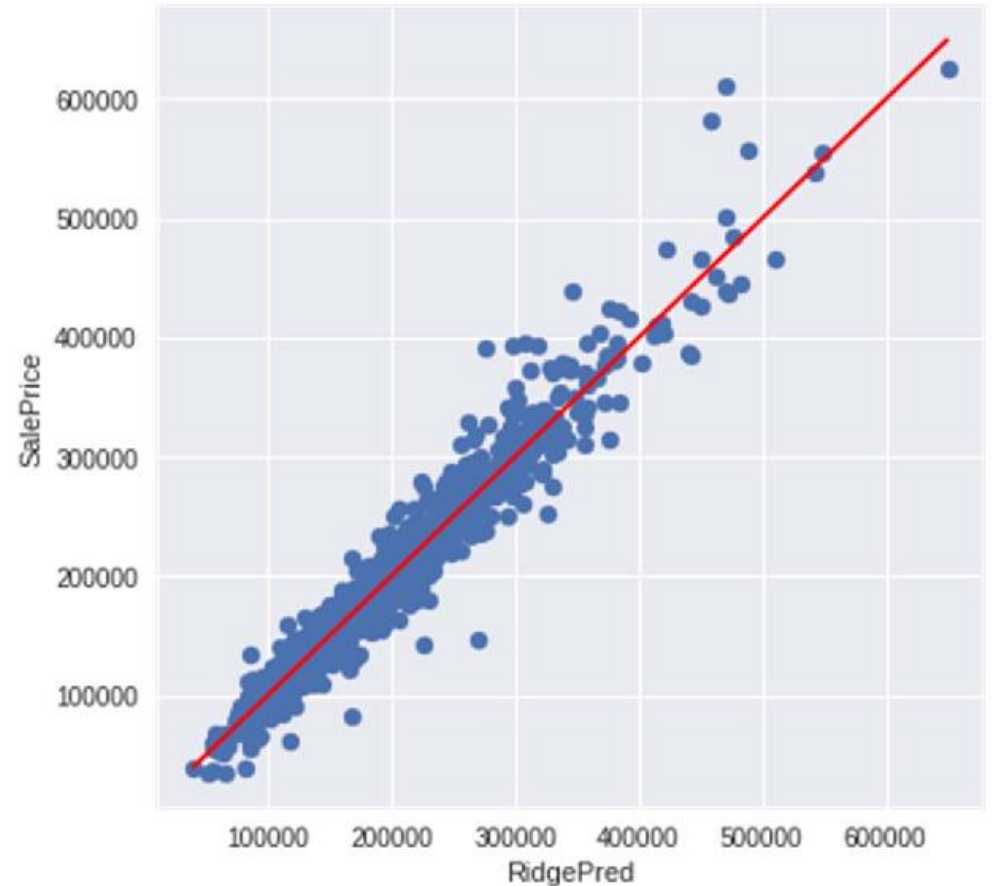


Fig. 4 Ridge Prediction for Training Data.

Causation vs correlation

Breakingviews

Zillow's failed house flipping

Reuters

WSJ NOV. 2021 : *“The company expects to record losses of more than \$500 million from home-flipping by the end of this year and is laying off a quarter of its staff.”*

Actual vs. predicted sale price of house

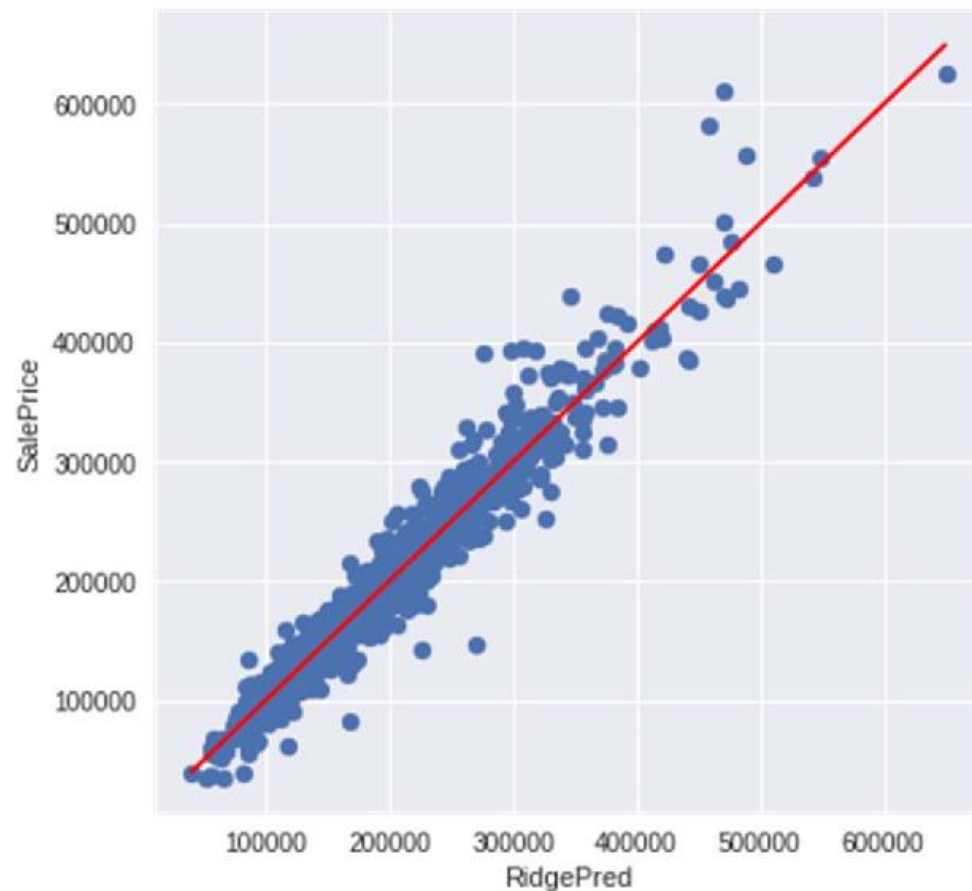


Fig. 4 Ridge Prediction for Training Data.